Mars Lander RL
How the code fits together

Version 2. New: Section 2 lists every **.env** variable and why it was kept. Then the call tree from **main()**, with every node explained in in-order (LVR) together with the data it receives from its caller and passes to the functions it calls.

Project	Viking lander (1976) terminal descent on Mars · MuJoCo · Stable-Baselines3
Version	2 (6 October 2026). Adds Section 2: the .env variables, why each was kept, the parked facts and where the other keys of the original 155 went. Sections 3–6 are version 1's Sections 2–5.
Milestone	M1 — simulator, viewer, video, scripted scenarios, 37 tests (no RL yet); simplified on 5 October 2026 (rigid legs, fixed CG, no roll jets, the agent cuts the engines)
Code	lander/ (13 modules) · scripts/ (6) · tests/ (7 files + conftest) · 1 notebook
Tree	224 nodes: 131 functions explained once, 66 reference nodes, 16 external library calls, plus the entry, test and notebook nodes
Facts	41 values in .env — 26 published, 5 derived, 10 assumed; 8 more parked as comments (Section 2)
Versions	Python 3.13.5 · MuJoCo 3.14.0 · numpy 2.5.3
Date	6 October 2026

Contents

1 The project in one page 3

Conventions used everywhere . . . 3

2 The .env variables, and why each was kept 5

The 41 at a glance . . . 5

The decisions behind the cut (5 October 2026) . . . 5

The four tests a variable had to pass . . . 5

Reading the variable tables . . . 6

Engines (7): VIKING_ENGINE_* . . . 6

Mass and inertia (6): VIKING_MASS_* . . . 7

Legs (3): VIKING_LEG_* . . . 7

Aerodynamics (2): VIKING_AERO_* . . . 8

Radars (9): VIKING_RADAR_* . . . 8

Landing requirements (3): VIKING_REQ_* . . . 9

Reference (1): VIKING_REF_* . . . 9

Mars (2): MARS_* . . . 9

Wind (1): MARS_WIND_* . . . 9

Terrain (3): MARS_TERRAIN_* . . . 10

Simulation (4): SIM_* . . . 10

Closest calls . . . 10

Parked: in .env as comments (8) . . . 11

Where the other keys went . . . 11

Moved into the code (23) . . . 11

Removed (79) . . . 12

3 How to read the tree 13

In-order (LVR) for nodes with any number of children . . . 13

What each entry in Section 6 contains . . . 13

4 The data that travels between nodes 14

Config (from .env) . . . 14

MuJoCo objects . . . 17

5 The logical tree 18

The complete call tree . . . 19

6 Every node, in-order (LVR) 22

Appendix A Functions on no call path from main 48

Appendix B Where each file's functions are explained 48

1 The project in one page

Mars Lander RL simulates NASA's Viking lander (1976) in the final few hundred metres of its landing on Mars, so that a reinforcement-learning agent can later learn to land it in calm air and in dust storms, and a fuzzer can search for the conditions where it fails. Milestone M1, the code this guide describes, is the simulator: physics, a simplified Viking vehicle (rigid legs that break above 3.34 m/s down or 1.22 m/s sideways, the CG fixed on the centre line, no roll jets, engines that the pilot shuts off), Mars air and steady wind, radar sensors with Viking's errors, a live 3-D viewer, MP4 export, scripted test scenarios and the test suite. There is no manual flight: commands always come from code (scripted policies now, the agent from M4).

Data moves in one direction (Figure 1). Every number lives in `.env` with its unit, status and source (Section 2 lists them all and why each was kept). `load_config` turns the file into a typed `Config`. `LanderSim` builds the geometry, generates the MuJoCo model from it and creates the backend-agnostic vehicle layer. An episode is `reset(seed, initial state, options)` followed by `step(Command)` every 40 ms; each step runs 20 physics steps of 2 ms (Figure 2) and returns a `StepResult`. The scripts, tests and notebook only consume StepResults.

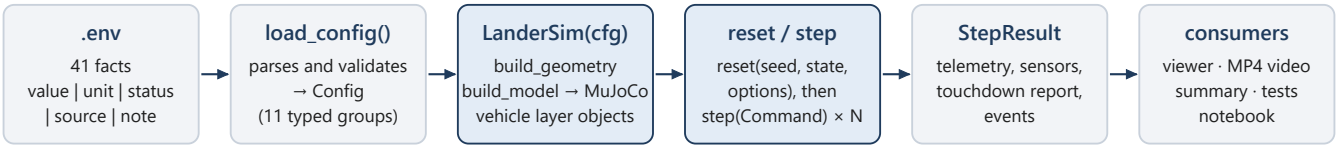


Figure 1 — the pipeline from facts to outputs.

Conventions used everywhere

- SI units. Angles are degrees in `.env` and in the API (`tilt_deg`), radians inside formulas.
- Lander frame (TN D-7959 axes): origin on the lander's centre line at the footpad plane, +Z up the thrust axis, leg 1 on +Y. The centre of gravity sits on this centre line, 0.79 m up, and never moves.
- World frame: origin at the landing target on the ground, +Z up. The ground is a plane through the origin, tilted for slope runs.
- MuJoCo state: `qpos` (7) = lander frame origin (3, world) and attitude quaternion (4, w x y z). `qvel` (6) = origin velocity (3, world) and angular velocity (3, body axes). The legs are rigid, so there are no joints.
- Throttle = Viking valve setting u in [0, 1]: thrust = 276 N + u × (2,667 − 276) N, so u = 0 is minimum thrust, not off. Engines stop only through the engines-off command (permanent) or an empty tank.
- Determinism (D10): one seed spawns two random streams (engine misalignment, radar noise) in a fixed order, and MuJoCo is deterministic, so the same seed, config and commands reproduce an episode bit for bit on the same machine.
- The vehicle layer (`config`, `geometry`, `vehicle`, `aero`, `sensors`, `touchdown`) is plain numpy and never imports MuJoCo (D1); `sim.py` connects it to MuJoCo.

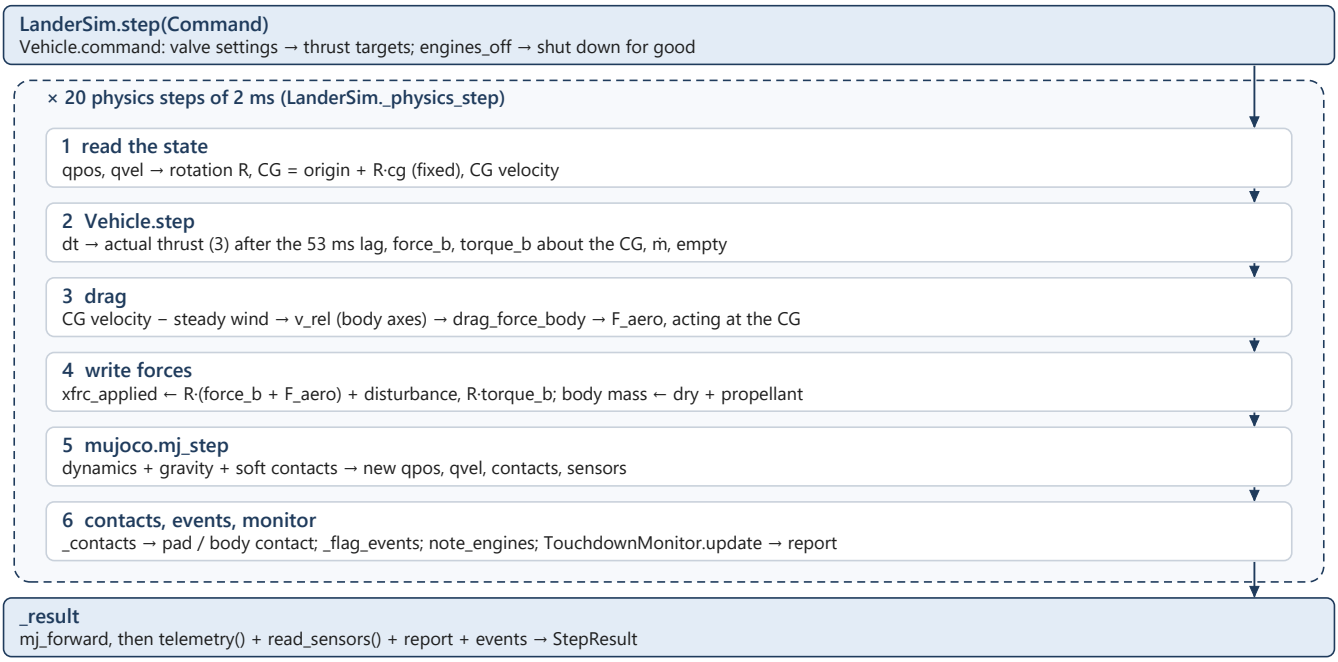


Figure 2 — one control step: the data passed between the stages of LanderSim.step and LanderSim._physics_step.

2 The .env variables, and why each was kept

`.env` is the only place numbers enter the simulator. On 5 October 2026 it was cut from 155 keys to 41: 26 published, 5 derived and 10 assumed. 8 more facts that later milestones need are parked as comments at the end of the file. This section lists all 41: what each one is, why it was kept and which functions read it. It ends with the closest calls, the parked facts, and where the other keys went. `docs/env_variables.md` has the same reasoning in Markdown.

The 41 at a glance

Group	Prefix	Keys	Published	Derived	Assumed
Engines	`VIKING_ENGINE_*`	7	4	2	1
Mass and inertia	`VIKING_MASS_*`	6	4	1	1
Legs	`VIKING_LEG_*`	3	3	0	0
Aerodynamics	`VIKING_AERO_*`	2	0	1	1
Radars	`VIKING_RADAR_*`	9	9	0	0
Landing requirements	`VIKING_REQ_*`	3	2	1	0
Reference	`VIKING_REF_*`	1	1	0	0
Mars	`MARS_*`	2	2	0	0
Wind	`MARS_WIND_*`	1	1	0	0
Terrain	`MARS_TERRAIN_*`	3	0	0	3
Simulation	`SIM_*`	4	0	0	4
Total		**41**	**26**	**5**	**10**

The decisions behind the cut (5 October 2026)

	Decision (brief number)	What it did to .env
D12	Rigid legs. A leg breaks if first contact is faster than 3.34 m/s down or 1.22 m/s sideways.	The two speed limits replace 22 leg values: crush struts, leg switches, load limiters, leg masses and the clearance and peak-load criteria.
D7	The agent commands engines off. The cut is permanent, and a landing counts only if the engines are off by the end of the episode.	Viking's leg switches (TESS) and their delay go; `engines_off` joins the actions.
D16	Episodes start at a random height of 200–500 m.	8 values for the parachute, ignition and handover go; START_PROPELLANT_KG is new.
D17	One agent learns calm and dust-storm landings from mixed episodes and is not told which.	The storm range stays; calm winds and gusts go.
D15	The CG sits on the centre line and never moves; mass = dry + propellant; inertia constant; ideal IMU at the CG; one altimeter; constant air density; one steady wind.	CG offsets, touchdown mass, IMU errors and position, the second altimeter, density spread and scale height go.
D4	Thrust misalignment stays; the roll jets go, so spin is not controlled.	7 roll-jet values and the roll-rate limit go.
D15	Viking's radar errors are on by default (settles the noise part of O4).	SENSOR_NOISE_SCALE = 1.
D5	One engine lag of 53 ms; no rate limit, no lag scale.	The rise and fall lags merge into T90_S; the rate limit and LAG_SCALE go.

The four tests a variable had to pass

1. **It acts.** The code reads it, and it changes the physics, the sensors or how a landing is judged. Reference-only values were dropped, except NASA's 45° tip-over figure, which the slope test compares against.
2. **It is not a duplicate.** No other variable already carries its effect. Dropped for this: the engine rate limit (repeats the lag), `LAG_SCALE` (repeats overriding the lag), the air-density spread (inside the storm range), and separate drag coefficients and areas (only their product is used).
3. **It is big enough to matter.** Each candidate was measured in 30 s of open-loop hover (engines held at a fixed balance, no controller) against the landing limits of 1.22 m/s sideways and 3.34 m/s down. Calm wind (< 0.08 m/s of drift), gusts (≤ 0.09 m/s) and the IMU errors (10^4–10^6× smaller than the signals) failed; misalignment (4–55° of tilt) passed easily.
4. **A published number beats an invented one.** Where the simulation needs some value anyway, such as a foot radius or a beam angle, the published one stays even when its effect is small.

Reading the variable tables

Variable shows the key without its group prefix, its value, then its status and source. A teal line marks a key that was merged or renamed in the cut. **Used in** lists the functions that read the value, linked to their entries in Section 6, and the test files that check it. `validate` (range checks) and `derived` (the numbers `check_config` prints) read most values, so they are listed only when nothing else does. Every number quoted comes from `.env` or from runs of the M1 code.

Engines (7): VIKING_ENGINE_*

Three fixed engines, throttled one by one: the agent's only actuators (no gimbal, no roll jets). A valve setting u from 0 to 1 gives 276 N + u × 2,391 N per engine. Only the engines-off command, which is permanent, or an empty tank gives zero thrust.

Variable	What it is	Why it is kept	Used in
THRUST_MIN_N **276 N** published · S2, S9	Thrust of one engine at its lowest lit setting (valve 0).	Viking could not throttle lower (10:1 range), so three lit engines always push at least 828 N, 35 % of the start weight. Only the engines-off command gives zero, which is why the last metres need a cut-off decision. It also limits steering near hover: there the engines run at 770–790 N, so each can be throttled down only about 500 N but up about 1,900 N.	[47] EngineBank.__init__
THRUST_MAX_N **2,667 N** published · S2, S9	Thrust of one engine at full setting (valve 1).	How hard the lander can brake: three engines give 3.4 × the start weight, a net deceleration of 8.9 m/s². From 30 m/s that stops the lander in about 50 m (ignoring the lag), so it sets how late braking can start. It also scales the plumes and the HUD's thrust percentages.	[47] EngineBank.__init__ [120] add_plumes [128] hud_lines
ISP_S **195 s** derived · S7, S8	Specific impulse: the engines' fuel economy.	Propellant flow = thrust ÷ (Isp × 9.807 m/s²). Hovering at the start burns 1.23 kg/s, so the 60 kg load lasts 49 s of hover or 14 s at full thrust. It turns every throttle choice into a fuel cost and decides when the tank runs dry, which fails the landing if the lander is still in the air.	[47] EngineBank.__init__
X_M **0.835, −0.835, 0.0 m** published · S4 Fig 6a	Sideways (X) position of each engine: +0.835 and −0.835 m for engines 1 and 2, 0 for engine 3.	There is no gimbal, so unequal throttles are the only way to tilt the lander. Engines 1 and 2 sit 0.835 m either side of the centre line: 100 N more on one than the other gives 83.5 N·m about Y, an angular acceleration of 22 °/s².	[32] build_geometry [47] EngineBank.__init__ tests: test_config
Y_M **0.482, 0.482, −0.989 m** published · S4 Fig 6a	Fore-aft (Y) position of each engine: 0.482 m for engines 1 and 2, −0.989 m for engine 3.	Engine 3 working against engines 1 and 2 tilts the lander about X: 100 N more on engine 3 gives 98.9 N·m, 35 °/s². The published positions are slightly unequal (2 × 0.482 ≠ 0.989), so equal throttles at hover leave a 20 N·m twist and engine 3 must run 2.5 % lower. `hover_trim` does this; the agent has to learn the same offset.	[32] build_geometry
T90_S **0.053 s** derived · S4 Fig 6b, S7 merged: T90_RISE_S 0.050 + T90_FALL_S 0.053	Time for an engine to complete 90 % of any thrust change, up or down, including the tail-off after engines-off (a first-order lag, τ = 23 ms).	The agent decides every 40 ms, but each command is only 82 % complete by the next decision, so it has to lead its commands rather than react. After the cut, thrust needs 53 ms to fall to 10 %, so the lander is still being braked for a moment after it touches. One value replaces the separate rise and fall lags (50 and 53 ms) and the rate limit, which repeated it.	[47] EngineBank.__init__ tests: test_config, test_physics
MISALIGN_DEG **0.044°** assumed · S7 /c	Each engine's thrust points 0.044° off its axis, in a random direction drawn every episode.	The only steady disturbance in calm air. Uncorrected for 30 s it tilts the lander 4–55° (median 19°, 20 seeds), pushes it 3–34 m/s sideways and spins it at 2–9 °/s. The direction changes every episode, so no fixed trim can cancel it: the agent must learn real attitude feedback. Without it every calm episode would be the same, and one fixed trim would hold the lander level (the hover test stays at 0.000° of tilt).	[47] EngineBank.__init__

Mass and inertia (6): VIKING_MASS_*

Mass = dry mass + propellant left, updated every physics step. The CG and the inertia never change (D15).

Variable	What it is	Why it is kept	Used in
DRY_KG **573.1 kg** derived · S2, S3 p141 merged: FUELLED_KG 657 – PROPELLANT_KG 83.9	The lander with empty tanks: 657 kg fuelled (S2) minus 83.9 kg of terminal-descent propellant (S3).	Mass at every physics step = 573.1 kg + the propellant left. As the tank empties, the weight falls by up to 9.5 % (2,349 to 2,126 N) and the thrust needed to hover falls with it, so the agent must keep adjusting.	[33] MassModel.__init__ tests: test_model
START_PROPELLANT_KG **60 kg** assumed · our choice, no source new key	Propellant on board when an episode starts, 200–500 m up.	The fuel budget: 49 s of hovering at the start weight. It decides whether a slow, careful descent is affordable. Assumed: our estimate is that a careful landing from 500 m at 30 m/s needs about 45–55 kg, and the M3 classical controller will check it. New, because episodes no longer start at engine ignition with the full 83.9 kg.	[34] build_mjcf [57] LanderSim. state_above_ground [72] LanderSim.reset tests: test_model, test_physics
IXX_KGM2 **162.7 kg·m²** published · S4 Fig 3	Moment of inertia about the body X axis (S4, at touchdown mass).	How quickly a torque about X turns the lander. That is the axis engine 3 controls against engines 1 and 2. Held constant because the propellant is at most 9.5 % of the mass.	[33] MassModel.__init__ tests: test_model
IYY_KGM2 **216.9 kg·m²** published · S4 Fig 3	Moment of inertia about the body Y axis.	How quickly a torque about Y turns the lander. That is the axis engines 1 and 2 control by throttling against each other.	[33] MassModel.__init__ tests: test_model
IZZ_KGM2 **298.3 kg·m²** published · S4 Fig 3	Moment of inertia about the thrust axis (Z).	Nothing controls rotation about Z (there are no roll jets), so this alone sets how fast the misalignment spins the lander: 2–9 °/s after 30 s.	[33] MassModel.__init__ tests: test_model
CG_Z_M **0.7904 m** published · S4 Fig 3	Height of the centre of gravity above the footpad plane, on the centre line.	Sets how easily the lander tips: CG height ÷ footpad radius = 0.566 (S4). Resting on a slope, it tips once the slope passes 41.5° with an edge of the footprint facing downhill (atan of 0.70 m ÷ 0.79 m). It is also the 0.26 m lever arm through which the misaligned thrust tilts the lander.	[32] build_geometry tests: test_model

Legs (3): VIKING_LEG_*

Rigid legs ending in three contact spheres. The break rule has no variables of its own: it uses the two speed limits under Landing requirements.

Variable	What it is	Why it is kept	Used in
AZIMUTH_DEG **90°, −30°, 210°** published · S4 Fig 3	Directions of the three legs around the centre line (leg 1 on +Y).	With the footpad radius it places the three feet: the triangle the lander stands on and tips over, and the edges the stability distance is measured to. The published directions put the legs between the engines, as on Viking; without them we would have to invent positions.	[32] build_geometry tests: test_model
FOOTPAD_RADIUS_M **1.397 m** published · S4 Fig 3, S5 Fig 8	Distance from the centre line to each foot (55.00 in).	The size of the footprint. Its edges lie 0.70 m from the centre, and that distance is the tip-over margin behind the stability check and the 41.5° static limit.	[32] build_geometry tests: test_model
FOOTPAD_Z_M **0.03175 m** published · S4 Fig 3, S5 Fig 8	Height of each foot's junction above the ground plane, used as the contact-sphere radius (3.2 cm).	Where the feet meet the ground. MuJoCo needs a contact radius, and the footpad altitude and the moment of touchdown depend on it. Its effect is small; it stays because the published size beats an invented one.	[32] build_geometry tests: test_model

Aerodynamics (2): VIKING_AERO_*

Drag on each body axis = ½ × air density × Cd·A × (relative speed)^2, where relative speed = lander velocity − wind; it acts at the CG. Cd·A is the drag coefficient times the area facing the air. Only the product is ever used, so each axis has one number.

Variable	What it is	Why it is kept	Used in
CDA_AXIAL_M2 **1.80 m²** assumed · S2 merged: CD_AXIAL 1.05 × AREA_AXIAL_M2 1.71	Cd·A for air moving along the thrust axis: Cd 1.05 (assumed) × the 1.71 m² body outline (S2).	Drag from the descent itself, plus the axial part of a storm acting on a tilted lander. Small: 13 N at 30 m/s, 0.55 % of the weight. One of the closest calls.	[54] drag_areas
CDA_LATERAL_M2 **2.01 m²** derived · S4 Table II, S3 p98, S11 merged: CD_LATERAL 1.0 × AREA_LATERAL_M2 2.01	Cd·A for air moving sideways, calibrated to NASA's numbers.	**The storm's push.** A 17–30 m/s storm pushes with 4.6–14.5 N, which drifts an uncorrected lander 0.22–0.69 m/s in 30 s: up to 57 % of the 1.22 m/s sideways limit. Calibrated so that a 51 m/s wind in 0.022 kg/m³ air gives NASA's 57.58 N (S3, S4). Without it a storm would have no effect.	[54] drag_areas

Radars (9): VIKING_RADAR_*

Viking's radar altimeter and four-beam Doppler radar, both antennas at the centre of the underside. The published accuracies are treated as 3σ (assumed). `SIM_SENSOR_NOISE_SCALE` switches the noise on (1, the default) or off (0).

Variable	What it is	Why it is kept	Used in
PERIOD_S **0.2 s** published · S3 p93 merged: RADALT_PERIOD_S + TDLR_PERIOD_S	Both radars give a new reading every 0.2 s and hold it in between.	The agent decides 25 times a second but gets fresh radar data only 5 times a second, so it must act on readings up to 0.2 s old. At full braking (8.9 m/s²) a held velocity can be 1.8 m/s out of date. One value replaces the two equal radar periods.	[109] SensorSuite.sample
ALT_MIN_RANGE_M **41 m** published · S3 p103, p109 was RADALT_MIN_RANGE_M	Altimeter readings below 41 m are thrown away.	Viking ignored them because of false echoes off its own structure. So the last 41 m, the riskiest part, have no altitude reading at all: the agent must carry its height forward using the Doppler velocity and the IMU.	[111] SensorSuite._read_altimeter
ALT_ERR_FRAC **0.045** published · S3 p109 was RADALT_ERR_FRAC	The part of the altimeter error that grows with range: 4.5 % (3σ).	Dominates high up. The 1σ error = (4.5 % × range + 1.5 m) ÷ 3 is 8 m at 500 m and 2 m at 100 m, so the starting height is known only to within metres.	[111] SensorSuite._read_altimeter tests: test_aero_sensors
ALT_ERR_M **1.5 m** published · S3 p109 was RADALT_ERR_M	The fixed part of the altimeter error: 1.5 m (3σ).	Dominates low down: together with ALT_ERR_FRAC the error is 1.1 m (1σ) at 41 m, the last valid reading.	[111] SensorSuite._read_altimeter tests: test_aero_sensors
DOPPLER_SPLAY_DEG **21.5°** published · S3 p112 was TDLR_SPLAY_DEG	Angle of each of the four Doppler beams from straight down.	Decides which beams still hit the ground when the lander tilts, and how the beam speeds combine into a velocity. Each beam sees only cos 21.5° = 93 % of a vertical descent, so in a straight, level descent all four lose lock below 1.07 m/s.	[32] build_geometry
DOPPLER_CLOCK_DEG **45°** published · S3 Fig 60 was TDLR_CLOCK_DEG	Rotation of the beams about the thrust axis: 45° from the pitch and yaw axes.	Fixes the beam directions in the body, and so which beam drops out first as the lander tilts about a given axis. One of the closest calls: the uncontrolled spin turns the beams anyway.	[32] build_geometry
DOPPLER_MIN_SPEED_MPS **1 m/s** published · S3 p112 was TDLR_MIN_SPEED_MPS	A beam loses lock when its closing speed falls below 1 m/s.	No velocity reading near hover. If the agent slows below about 1 m/s of descent before touchdown, it loses its speed measurement, so it cannot creep down slowly on radar data alone.	[114] SensorSuite._read_doppler
DOPPLER_ERR_FRAC **0.015** published · S3 p112 was TDLR_ERR_FRAC	The part of the velocity error that grows with speed: 1.5 % (3σ).	Dominates above 20 m/s: the 1σ error = max(1.5 % × speed, 0.3 m/s) ÷ 3 is 0.15 m/s at 30 m/s, early in the descent.	[114] SensorSuite._read_doppler tests: test_aero_sensors
DOPPLER_ERR_MPS **0.3 m/s** published · S3 p112 was TDLR_ERR_MPS	The smallest velocity error: 0.3 m/s (3σ).	Dominates below 20 m/s, so for the whole final approach the error is 0.1 m/s (1σ): 8 % of the 1.22 m/s sideways limit, and the main sensor error near touchdown.	[114] SensorSuite._read_doppler tests: test_aero_sensors

Landing requirements (3): VIKING_REQ_*

These decide success: no broken leg, no tip-over, no body strike, settled, engines off by the end of the episode, and propellant left at touchdown. The pad radius and tilt limit come at M2 (open decision O3).

Variable	What it is	Why it is kept	Used in
`VERTICAL_SPEED_MPS` **3.34 m/s** derived · S5 Table 1	Fastest allowed descent speed at first footpad contact: Viking's 2.44 m/s design speed + 0.9 m/s.	Any faster and a leg breaks (Viking's leg design limit, S5), failing the landing on the spot. It defines how gently the agent must arrive. As the rigid-leg break rule it replaces the 22 removed leg values (crush struts, leg switches, load limiters, leg masses and the old leg criteria).	[93] TouchdownMonitor._judge [167] TouchdownMonitor.update tests: test_legs
`HORIZONTAL_SPEED_MPS` **1.22 m/s** published · S5 Table 1	Fastest allowed sideways speed at first footpad contact.	Any faster and a leg breaks (S5). This is the budget that storm drift (up to 0.69 m/s uncorrected) and Doppler error (0.1 m/s) eat into.	[93] TouchdownMonitor._judge [167] TouchdownMonitor.update tests: test_legs
`SETTLE_KE_FRAC` **0.005** published · S4 p9	Settled = resting on all three feet with kinetic energy below 0.5 % of the impact energy, held for 0.5 s.	Ends the episode and lets the landing be judged (S4's criterion). Without it there is no clear moment when a landing is over.	[167] TouchdownMonitor.update

Reference (1): VIKING_REF_*

Published results the simulation is compared with. Nothing in the physics reads them.

Variable	What it is	Why it is kept	Used in
`TIPOVER_P_45DEG` **0.85** published · S4 p15	NASA's probability that Viking tips over landing on a 45° slope (with crushable legs).	A comparison target, not physics: the slope test prints our rate next to it. With rigid legs on μ = 0.6 ground ours is 1.00 (24 drops), because the feet slide; at 10–30° it is 0.00. One of the closest calls.	tests: test_legs

Mars (2): MARS_*

Both are held constant over the last 500 m.

Variable	What it is	Why it is kept	Used in
`GRAVITY_MPS2` **3.71 m/s²** published · S11	Mars surface gravity.	The weight the engines fight: 2,349 N at the start mass. It sets the hover thrust and the free-fall speed, and gives the free-fall test its exact answer.	[34] build_mjcf [83] LanderSim.hover_trim tests: test_config, test_physics
`AIR_DENSITY_KGPM3` **0.016 kg/m³** published · S11	Air density, used at every height (it changes by only 4.4 % over 500 m).	Scales every aerodynamic force, since drag is proportional to density. Thin Mars air is why even a 30 m/s storm pushes with only 14.5 N. The ±0.006 spread was removed because the storm range already covers it.	[30] LanderSim.__init__ tests: test_aero_sensors

Wind (1): MARS_WIND_*

One steady wind per episode: none in calm episodes, a storm wind from this range otherwise. Calm winds and gusts were measured and removed (see “Where the other keys went”).

Variable	What it is	Why it is kept	Used in
`STORM_MPS` **17, 30 m/s** published · S11 was DUSTSTORM_MPS	Dust-storm wind speeds at the Viking sites (S11).	Defines the storm half of training: M2 will draw a steady storm wind from 17–30 m/s in about 30 % of episodes. In M1 only the storm-push estimate in `derived` and the tests read it; the scripts take `--wind`.	[22] derived tests: test_aero_sensors

Terrain (3): MARS_TERRAIN_*

The ground is a tilted plane through the target. These are the defaults a reset uses when its options give no slope or friction.

Variable	What it is	Why it is kept	Used in
SLOPE_DEG **0°** assumed · our choice, no source	Default ground slope.	The ground every reset uses unless its options say otherwise (flat). M2 will draw slopes from the parked distribution (90 % below 11.7°).	[57] LanderSim. state_above_ground [72] LanderSim.reset
SLOPE_DIRECTION_DEG **0°** assumed · our choice, no source	Default downhill direction, measured from +X toward +Y.	Together with the lander's heading it decides which edge of the footprint faces downhill, and so how easily the lander tips.	[57] LanderSim. state_above_ground [72] LanderSim.reset
FRICTION **0.6** assumed · S4, S5	Default footpad friction coefficient, the middle of the published 0.2–1.0.	Decides whether the feet grip or slide after contact. On a 45° slope μ = 0.6 is below tan 45° = 1, so the feet slide and every drop tips over. M2 will draw it from 0.2–1.0.	[34] build_mjcf [72] LanderSim.reset

Simulation (4): SIM_*

Numerical settings, not physics. All four are assumed.

Variable	What it is	Why it is kept	Used in
PHYSICS_DT_S **0.002 s** assumed · brief	MuJoCo's integration step.	Small enough for stable contact with rigid legs and the 5 ms contact time constant, which must span at least 2 steps. Twenty physics steps make one control step.	[16] SimCfg.substeps [30] LanderSim.__init__ [34] build_mjcf tests: test_physics
CONTROL_HZ **25 Hz** assumed · brief, S3 p105	Agent decisions per second; each command is held for 40 ms.	Matches Viking's 40 ms guidance cycle (S3). With the 53 ms engine lag and the 0.2 s radar period it sets how quickly the agent can react. Also the video frame rate.	[15] SimCfg.control_dt_s [26] export_video.main [69] VideoWriter.__init__
CONTACT_TIMECONST_S **0.005 s** assumed · our choice, no source	How soft MuJoCo makes the ground contact (its time constant).	With rigid legs this alone shapes the impact: a 2.44 m/s touchdown rebounds at about 0.29 m/s. It must stay at least 2 physics steps.	[34] build_mjcf
SENSOR_NOISE_SCALE **1.0** assumed · brief	Radar-noise switch: 1 = Viking's published errors, 0 = perfect readings.	On by default (decided on 5 October). Tests set 0 when they need exact readings, and a reset option can change it per episode, for curricula or fuzzing.	[50] SensorSuite.reset [51] SensorSuite.__init__ tests: test_aero_sensors

Closest calls

The three variables with the weakest case. Removing any of them would barely change the simulation; they are listed so the choice stays visible.

Variable	If it were removed	Why it stays
VIKING_REF_TIPOVER_P_45DEG	Nothing in the simulation. The value would move into the slope test as a constant with its source.	.env is where every published number lives with its source (D14), and the README's validation table quotes it.
VIKING_AERO_CDA_AXIAL_M2	Using the lateral 2.01 m^2 on every axis would raise the axial drag by 12 %: about 1.5 N at 30 m/s, under 0.1 % of the weight.	It is the only number for drag along the thrust axis; merging would apply a value calibrated for sideways flow.
VIKING_RADAR_DOPPLER_CLOCK_DEG	Setting the beams to 0° would change only which beam drops out first as the lander tilts; the spin turns the beams in any case.	The beams need some angle, and Viking's is published (S3 Fig 60).

Parked: in .env as comments (8)

Facts that a later milestone will read. `parse_env_file` skips comment lines, so they cost nothing now. To use one, remove its leading '# ' and add its field (and group) to `lander/config.py`.

Key	Value	Needed at	What it will do
`VIKING_SEQ_TERMINAL_SPEED_MPS`	**2.44 m/s** published · S3 p104	M3	Viking's constant final descent speed; the classical controller's last phase follows it.
`VIKING_SEQ_TERMINAL_ALT_M`	**16.8 m** published · S3 p104	M3	Height where that constant-speed phase begins.
`VIKING_SEQ_AXIAL_THROTTLE_LIMIT`	**0.84** published · S3 p99	M3	Average valve limit for the controller's descent-speed loop.
`VIKING_REQ_TILT_DEG`	**5°** published · S3 Table 8	M2 (O3)	Candidate tilt limit at touchdown.
`VIKING_REQ_RATE_PITCHYAW_DPS`	**7 °/s** published · S3 Table 8	M2 (O3)	Candidate turn-rate limit at touchdown.
`MARS_TERRAIN_SLOPE_P90_DEG`	**11.7°** published · S4 p15, Table II	M2	Slope distribution for training resets: exponential, 90 % below 11.7°.
`MARS_TERRAIN_FRICTION_RANGE`	**0.2, 1.0** published · S4 p8, S5 Table 1	M2	Friction range for training resets.
`VIKING_REQ_STABILITY_PROB`	**0.997** published · S5 Table 1	M5	Viking's landing-gear stability: a benchmark for the evaluation.

Where the other keys went

The 155-key file of 5 October 2026 (before the cut), key by key. The script that made this guide checks that every old key appears exactly once below or in the tables above.

What happened	Old keys	In the new .env
Kept unchanged	26	26 live keys
Merged or renamed	19	14 live keys, marked in teal in their rows
New	–	1 live key: START_PROPELLANT_KG
Parked as comments	8	8 comment lines (see Parked)
Moved into the code	23	constants and a test (first table below)
Removed	79	by theme (second table below)
Total	**155**	**41 live keys + 8 parked lines**

Moved into the code (23)

Values that only shape the drawing, or cannot change, became named constants with their sources in a comment.

Old keys	Where the value lives now	File
`VIKING_ENGINE_COUNT, VIKING_LEG_COUNT`	Implied by the lists: `validate` requires 3 engine coordinates and 3 leg directions.	`lander/config.py`
`VIKING_ENGINE_Z_M, VIKING_BODY_BOTTOM_Z_M`	`BODY_BOTTOM_Z_M` = 0.5265 m: the engines and both radar antennas sit on the underside.	`lander/geometry.py`
`VIKING_TDLR_BEAMS`	`DOPPLER_BEAMS` = 4.	`lander/geometry.py`
`VIKING_BODY_LONG_SIDE_M, VIKING_BODY_SHORT_SIDE_M, VIKING_BODY_HEIGHT_M, VIKING_BODY_TDLR_DROP_M, VIKING_BODY_TDLR_SIZE_M, VIKING_MASS_TANK_AZIMUTH_DEG, VIKING_MASS_TANK_RADIUS_M, VIKING_MASS_TANK_Z_M, VIKING_MASS_TANK_DIAMETER_M, VIKING_LEG_FOOTPAD_DIAMETER_M, VIKING_LEG_STRUT_TOP_RADIUS_M, VIKING_LEG_STRUT_TOP_Z_M, VIKING_LEG_BIPOD_RADIUS_M, VIKING_LEG_BIPOD_HALF_SPAN_M, VIKING_LEG_BIPOD_Z_M`	Drawing constants `BODY_*`, `RADAR_HOUSING_M`, `TANKS`, `FOOTPAD_DIAMETER_M`, `STRUT_TOP` and `BIPOD`, each with its source. They shape the model you see; the hull, radar housing, engines and tanks also count for body strikes.	`lander/mjcf.py`
`SIM_G0_MPS2`	`G0_MPS2` = 9.80665: standard gravity is a definition, not a setting.	`lander/config.py`
`SIM_CONTACT_DAMPRATIO`	The 1 in the contact `solref`: MuJoCo's default damping ratio.	`lander/mjcf.py`
`VIKING_REF_CG_HEIGHT_RATIO`	The test asserts CG height ÷ footpad radius = 0.566.	`tests/test_model.py`

Removed (79)

Theme	Old keys	Why they went
Roll jets and the roll-rate limit (8)	`VIKING_ROLLJET_COUNT, VIKING_ROLLJET_THRUST_N, VIKING_ROLLJET_ISP_S, VIKING_ROLLJET_ARM_M, VIKING_ROLLJET_Z_M, VIKING_ROLLJET_VALVE_DELAY_S, VIKING_ROLLJET_MIN_ON_S, VIKING_REQ_RATE_ROLL_DPS`	D4: spin is not controlled. The misalignment spins the lander 2–9 °/s over 30 s, which moves the feet at most 0.22 m/s at touchdown: harmless with rigid legs. The actions shrink to 3 throttles + engines off.
Crushable legs, leg switches (TESS) and the leg criteria (22)	`VIKING_STRUT_PRESTROKE_FORCE_N, VIKING_STRUT_CRUSH_ONSET_N, VIKING_STRUT_CRUSH_RAMP_M, VIKING_STRUT_CRUSH_PLATEAU_N, VIKING_STRUT_PLATEAU_STROKE_M, VIKING_STRUT_TENSION_N, VIKING_STRUT_STIFFNESS_NPM, VIKING_STRUT_DAMPING_NSPM, VIKING_STRUT_SCATTER_3SIGMA, VIKING_TESS_TRIP_STROKE_M, VIKING_TESS_DELAY_MEAN_S, VIKING_TESS_DELAY_3SIGMA_S, VIKING_MASS_LEG_MOVING_KG, VIKING_MASS_GEAR_KG, VIKING_REQ_STRUT_STROKE_M, VIKING_REQ_PEAK_LOAD_G, VIKING_REQ_CLEARANCE_M, VIKING_REF_STROKE_3SIGMA_M, VIKING_REF_LIMITER_YIELD_N, VIKING_REF_STRUT_LENGTH_M, VIKING_REF_CLEARANCE_MEAN_M, VIKING_REF_CLEARANCE_3SIGMA_M`	D12 and D7: rigid legs that break above the two speed limits, and the agent, not leg switches, cuts the engines. A force limit on rigid legs would depend on MuJoCo's contact softness. The body cannot near the ground before a tip-over, so no clearance limit; peak g is set by the contact softness. The 20 kg of leg mass is part of the dry mass.
Moving CG (3)	`VIKING_MASS_TOUCHDOWN_KG, VIKING_MASS_CG_X_M, VIKING_MASS_CG_Y_M`	D15: the CG sits on the centre line (Viking's was 4 cm off). The touchdown mass only anchored the published CG and inertia values.
Viking's earlier flight phases (8)	`VIKING_SEQ_IGNITION_ALT_M, VIKING_SEQ_CHUTE_RELEASE_S, VIKING_SEQ_WARMUP_S, VIKING_SEQ_WARMUP_THROTTLE, VIKING_SEQ_HANDOVER_SPEED_MPS, VIKING_SEQ_HANDOVER_TILT_DEG, VIKING_SEQ_PROBABLE_SPEED_MPS, VIKING_SEQ_DESCENT_TIME_S`	D16: episodes start 200–500 m up, after the parachute, engine ignition and the handover to terminal guidance.
Limits never reached below 500 m (3)	`VIKING_RADALT_MAX_RANGE_M, VIKING_TDLR_MAX_RANGE_M, VIKING_TDLR_MAX_SPEED_MPS`	137 km, 6 km and 225 m/s; an episode starts at most 500 m up and falls at tens of m/s.
Engine settings with no effect, or that repeat the lag (4)	`VIKING_ENGINE_RATE_LIMIT_NPS, SIM_LAG_SCALE, VIKING_ENGINE_CANT_DEG, VIKING_ENGINE_NOISE_FRAC`	D5: the rate limit added 12 ms, only on full-range jumps, and its value was guessed from the lag; LAG_SCALE does what overriding the lag does. Cant and thrust noise were both 0 (no data).
IMU errors and position (7)	`VIKING_IMU_POS_M, VIKING_IMU_RATE_HZ, VIKING_IMU_GYRO_BIAS_DPH, VIKING_IMU_GYRO_SCALE_PPM, VIKING_IMU_ACCEL_BIAS_G, VIKING_IMU_ACCEL_SCALE_PPM, VIKING_IMU_MISALIGN_MRAD`	The IMU is ideal, sits at the CG and is read every control step. Its published errors are 10^4–10^6× smaller than the signals; the off-centre position only added up to ~10 m/s² of turn-induced acceleration (noise, not information).
Second altimeter and antenna positions (3)	`VIKING_RADALT_COUNT, VIKING_RADALT_POS_M, VIKING_TDLR_POS_M`	The two altimeters were identical, so one is modelled. Both antennas sit at the centre of the underside; the old positions were guesses (not dimensioned in the sources).
Calm wind, gusts, shear and air variations (13)	`MARS_WIND_SPEED_MPS, MARS_WIND_DIRECTION_DEG, MARS_WIND_GUST_SIGMA_MPS, MARS_WIND_GUST_TAU_S, MARS_WIND_SHEAR_EXPONENT, MARS_WIND_REF_HEIGHT_M, MARS_WIND_SUMMER_MPS, MARS_WIND_FALL_MPS, MARS_AIR_DENSITY_SIGMA_KGPM3, MARS_SCALE_HEIGHT_M, MARS_SURFACE_PRESSURE_PA, MARS_WIND_SURVIVAL_MPS, VIKING_AERO_COP_OFFSET_Z_M`	Calm winds (2–10 m/s) drift the lander < 0.08 m/s in 30 s and gusts ≤ 0.09 m/s; shear and the centre-of-pressure offset were 0. The density spread (+37 % push) sits inside the storm range (3× push); density changes 4.4 % over 500 m, and surface pressure was never used. The 90 m/s survival wind applies after landing.
Calibration inputs and NASA's touchdown results (8)	`MARS_WIND_DESIGN_99_MPS, VIKING_REF_TD_AERO_3SIGMA_N, VIKING_REF_TD_AERO_MEAN_N, VIKING_REF_TD_THRUST_N, VIKING_REF_TD_VZ_MPS, VIKING_REF_TD_VZ_3SIGMA_MPS, VIKING_REF_TD_VH_MPS, VIKING_REF_TD_VH_3SIGMA_MPS`	The 51 m/s design wind and NASA's 57.58 N aero force only calibrated CDA_LATERAL; their values sit in its note. The rest were outputs of NASA's own touchdown simulation that no code read; the thrusts also reflected the 4 cm CG offset (engine 3 ran 15 % low, now 2.5 %).

3 How to read the tree

- A node is a function, method, property or nested function. A few nodes are not functions: the root `main` (the program entry), the test suite and its files, the notebook, and calls into external libraries (MuJoCo, imageio).
- A node's children are what it calls, in the order it calls them.
- A function called from several places is explained once, at its home position. Every other call appears as a reference node (↺) that links to the home and states the data passed at that call site.
- Every parent → child edge was checked against the source code by the script that made this document, and the file:line next to each node comes from the code itself.

In-order (LVR) for nodes with any number of children

In-order visits a binary node as Left subtree, then the node (Visit), then the Right subtree. The functions here call any number of others, so in-order is applied in its usual general-tree form: L = the subtree of the first child, V = the node, R = the subtrees of the remaining children, left to right. The number in brackets on every node is its position in that walk, and Section 6 explains the nodes in exactly that numeric order. Read this way, each function comes right after the first thing it calls and right before the later steps it coordinates.

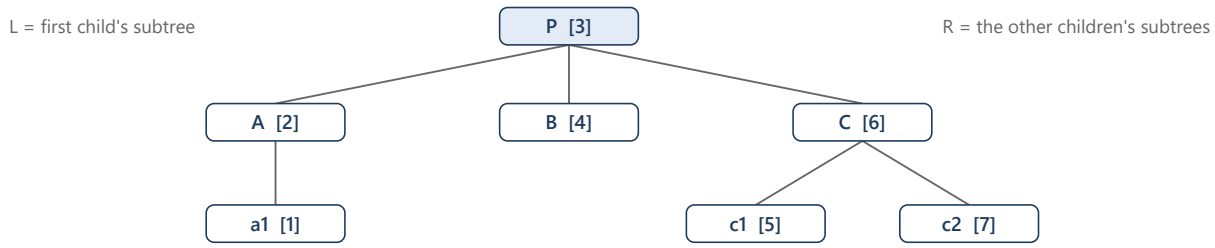


Figure 3 — in-order on a small tree: a1, A, P, B, c1, C, c2.

What each entry in Section 6 contains

- Header: in-order number, name with parameters, kind and file:line.
- Parent, L and R: links to the caller, the first callee and the other callees.
- What it does.
- Interface: **Receives** = the data its parent passes in; **Returns** = what goes back to the parent; **State** = data exchanged through objects instead of arguments (MuJoCo arrays, cached fields); **Calls** = for each child, the data passed down and the value that comes back.
- Reference (↺) and external nodes get a one-line entry with the data of that call.
- Every [number] is a link. The PDF's bookmark panel lists all nodes too.

4 The data that travels between nodes

These records are the vocabulary of every interface in Section 6.

Config (from .env)

`Config` holds one frozen dataclass per key prefix, plus `params` (key → Param) and `env_path`. Field name = key minus prefix, lower case: `VIKING_ENGINE_THRUST_MAX_N` → `cfg.engine.thrust_max_n`. `SimCfg` adds two properties, `control_dt_s` (0.04 s) and `substeps` (20).

prefix	group	keys	examples
`VIKING_ENGINE_*`	cfg.engine	7	thrust_min_n, thrust_max_n, isp_s, x_m …
`VIKING_MASS_*`	cfg.mass	6	dry_kg, start_propellant_kg, ixx_kgm2, iyy_kgm2 …
`VIKING_LEG_*`	cfg.leg	3	azimuth_deg, footpad_radius_m, footpad_z_m
`VIKING_AERO_*`	cfg.aero	2	cda_axial_m2, cda_lateral_m2
`VIKING_RADAR_*`	cfg.radar	9	period_s, alt_min_range_m, alt_err_frac, alt_err_m …
`VIKING_REQ_*`	cfg.req	3	vertical_speed_mps, horizontal_speed_mps, settle_ke_frac
`VIKING_REF_*`	cfg.ref	1	tipover_p_45deg
`MARS_WIND_*`	cfg.wind	1	storm_mps
`MARS_TERRAIN_*`	cfg.terrain	3	slope_deg, slope_direction_deg, friction
`MARS_*`	cfg.mars	2	gravity_mps2, air_density_kgpm3
`SIM_*`	cfg.sim	4	physics_dt_s, control_hz, contact_timeconst_s, sensor_noise_scale

`Param` lander/config.py:37

One fact: its value plus the metadata needed to verify it.

field	type	meaning
`key`	str	the .env key, e.g. VIKING_ENGINE_THRUST_MAX_N
`value`	Any	int, float or tuple of floats
`unit`	str	unit text from the .env comment
`status`	str	published | derived | assumed (| override)
`source`	str	where to check it: S-code and page / figure / table
`note`	str	free text; for an override: 'was <old value> (<status>; <source>)'
`origin`	str	'.env', 'env:LANDER_<KEY>', 'cli', 'code' or 'test'

`InitialState` lander/sim.py:80

Full initial state (brief section 9). Position/velocity are of the vehicle CG, world frame.

field	type	meaning
`position`	tuple	CG position, world frame (m)
`velocity`	tuple	CG velocity, world frame (m/s)
`quat`	tuple	attitude body → world (w, x, y, z)
`omega`	tuple	angular velocity in body axes (rad/s)
`propellant_kg`	float | None	propellant at the start; None = VIKING_MASS_START_PROPELLANT_KG (60 kg)
`throttle`	tuple | None	starting valve settings (3); None = hover trim

`ResetOptions` lander/sim.py:92

Per-episode environment conditions and faults (None = use the .env value).

field	type	meaning
`wind_speed_mps`	float	steady wind speed; 0 = calm (dust storms are 17–30 m/s)
`wind_direction_deg`	float	direction the wind blows toward
`slope_deg`	float | None	ground slope
`slope_direction_deg`	float | None	downhill direction
`friction`	float | None	footpad friction coefficient
`disturbance_force_n`	tuple	constant extra force, world frame (N)
`disturbance_torque_nm`	tuple	constant extra torque, body frame (N·m)
`engine_thrust_scale`	tuple	per-engine fault factor (1 nominal, 0 failed)
`engines_enabled`	bool	False = start with the engines already shut down (free fall)
`sensor_noise_scale`	float | None	None = the .env value (1 = Viking's radar errors, 0 = perfect readings)

`Command` lander/vehicle.py:30

Actuator command held for one control period (zero-order hold).

field	type	meaning
`throttle`	tuple[float, ...]	3 Viking valve settings 0 … 1 (0 = 276 N, 1 = 2,667 N per engine)
`engines_off`	bool	True shuts all three engines down for good (no relight)

`VehicleStep` lander/vehicle.py:101

field	type	meaning
`thrust`	np.ndarray	actual thrust per engine after the lag, clamp and faults (N)
`force_b`	np.ndarray	total thrust force, body frame (N)
`torque_b`	np.ndarray	thrust torque about the CG, body frame (N·m)
`mdot`	float	propellant flow this step (kg/s)
`empty`	bool	the tank has run dry

`Telemetry` lander/sim.py:120

Ground-truth snapshot at the end of a control step (for HUD, logs and debugging). Properties: vz, vh, spin_dps.

field	type	meaning
`t`	float	simulation time (s)
`cg`	np.ndarray	CG position, world (m)
`vel`	np.ndarray	CG velocity, world (m/s)
`quat`	np.ndarray	attitude (w, x, y, z)
`omega_b`	np.ndarray	body rates (rad/s)
`tilt_deg`	float	thrust axis from vertical
`height_cg`	float	CG height above the ground below it (m)
`pad_altitude`	float	lowest footpad above the ground (m; 0 at contact)
`propellant_kg`	float	propellant left
`mass_kg`	float	total mass = dry + propellant
`thrust_n`	np.ndarray	actual thrust per engine (N)
`throttle_cmd`	np.ndarray	last valve settings
`engines_off`	bool	engines shut down (command or empty tank)
`wind`	np.ndarray	the episode's steady wind, world (m/s)
`pad_contact`	tuple	footpad contact flags (3)
`body_contact`	bool	any body part touching the ground

`SensorReadings` lander/sensors.py:28

Properties: altitude_valid, doppler_valid.

field	type	meaning
`t`	float	sample time (s)
`accel`	np.ndarray	specific force at the CG, body frame (m/s²); ideal IMU
`gyro`	np.ndarray	body rates (rad/s); ideal IMU
`altitude`	float	radar-altimeter range to the nearest ground point (m), NaN = invalid (e.g. below 41 m)
`doppler_beam`	np.ndarray	4 Doppler beam closing speeds (m/s), NaN = no lock
`doppler_vel`	np.ndarray	body-frame ground velocity from ≥ 3 locked beams (m/s), NaN otherwise

`TouchdownReport` lander/touchdown.py:46

Properties: done, success.

field	type	meaning
`touched_down`	bool	a footpad or the body has touched
`first_contact_time`	float | None	time of first contact (s)
`vz_at_contact`	float | None	descent speed at first contact (m/s, + down)
`vh_at_contact`	float | None	horizontal speed at first contact (m/s)
`tilt_at_contact_deg`	float | None	tilt at first contact
`leg_broken`	bool	first contact faster than 3.34 m/s down or 1.22 m/s sideways
`min_stability_m`	float	minimum stability distance (m)
`settled`	bool	resting on all three feet, kinetic energy < 0.5 % of impact energy for 0.5 s
`settle_time`	float | None	time settled (s)
`tipped_over`	bool	stability distance went negative
`body_strike`	bool	a body part hit the ground
`propellant_exhausted_airborne`	bool	the tank ran dry before touchdown
`engines_off`	bool	engines shut down (must be True when the episode ends)
`final_tilt_deg`	float | None	tilt when the landing was decided
`final_xy`	tuple[float, float] | None	position when decided (m)
`criteria`	dict[str, bool | None]	name → True / False / None (None = decided at M2, O3)

`StepResult` lander/sim.py:155

field	type	meaning
`telemetry`	Telemetry	Telemetry (ground truth)
`sensors`	SensorReadings	SensorReadings
`report`	TouchdownReport	TouchdownReport (live object)
`events`	list[str]	this step's events: touchdown, leg_broken, engines_cut, settled, tipped_over, propellant_exhausted, body_strike

`Scenario` lander/scenarios.py:35

field	type	meaning
`name`	str	drop | hard_drop | slope45 | hover | freefall
`description`	str	one-line description
`duration_s`	float	maximum simulated time (s)
`initial_state`	InitialState	InitialState
`options`	dict	dict of ResetOptions fields
`policy`	Callable[[LanderSim], Command]	sim → Command (hover_then_cut; hover for the hover scenario)
`seed`	int	random seed

`LanderGeometry` lander/geometry.py:37

Everything positional about the lander, in the lander frame.

field	type	meaning
`footpads`	np.ndarray	footpad contact-sphere centres (3, 3)
`footpad_radius`	float	contact-sphere radius = junction height (0.03175 m)
`leg_azimuth_rad`	tuple	leg directions (3)
`cg`	np.ndarray	centre of gravity on the centre line (0, 0, 0.7904 m)
`engine_pos`	np.ndarray	engine thrust points on the underside (3, 3)
`radar_pos`	np.ndarray	altimeter and Doppler antennas: centre of the underside
`doppler_beams`	np.ndarray	beam directions (4, 3), body frame

`ModelIndex` lander/mjcf.py:38

MuJoCo ids/addresses the simulator needs every step.

field	type	meaning
`lander_body`	int	body id
`pad_geoms`	tuple[int, ...]	footpad sphere geom ids
`body_geoms`	tuple[int, ...]	hull, radar housing, engine and tank geom ids
`terrain_geom`	int	ground plane geom id
`ground_mocap`	int	mocap index of the ground body
`engine_sites`	tuple[int, ...]	engine site ids
`imu_acc_adr`	int	sensordata address of the IMU accelerometer
`imu_gyro_adr`	int	sensordata address of the IMU gyro

MuJoCo objects

- `MjModel` (model): constants compiled from the generated XML. The simulator writes the lander's `body_mass` and `body_subtreemass` as propellant burns (the CG and inertia never change), and the terrain's `geom_friction` at reset.
- `MjData` (data): the state. `qpos`, `qvel` (layout in Section 1), `time`, `mocap_quat` (ground tilt), `xfrc_applied` (force and torque on the lander body, written every physics step), `contact` (read by `_contacts`), `sensordata` (IMU accelerometer and gyro at the CG).

5 The logical tree

Figure 4 shows the top three levels. The complete tree follows: every node with its in-order number (a link to its entry), the file and line where it is defined, and ↺ for references to a node explained elsewhere.

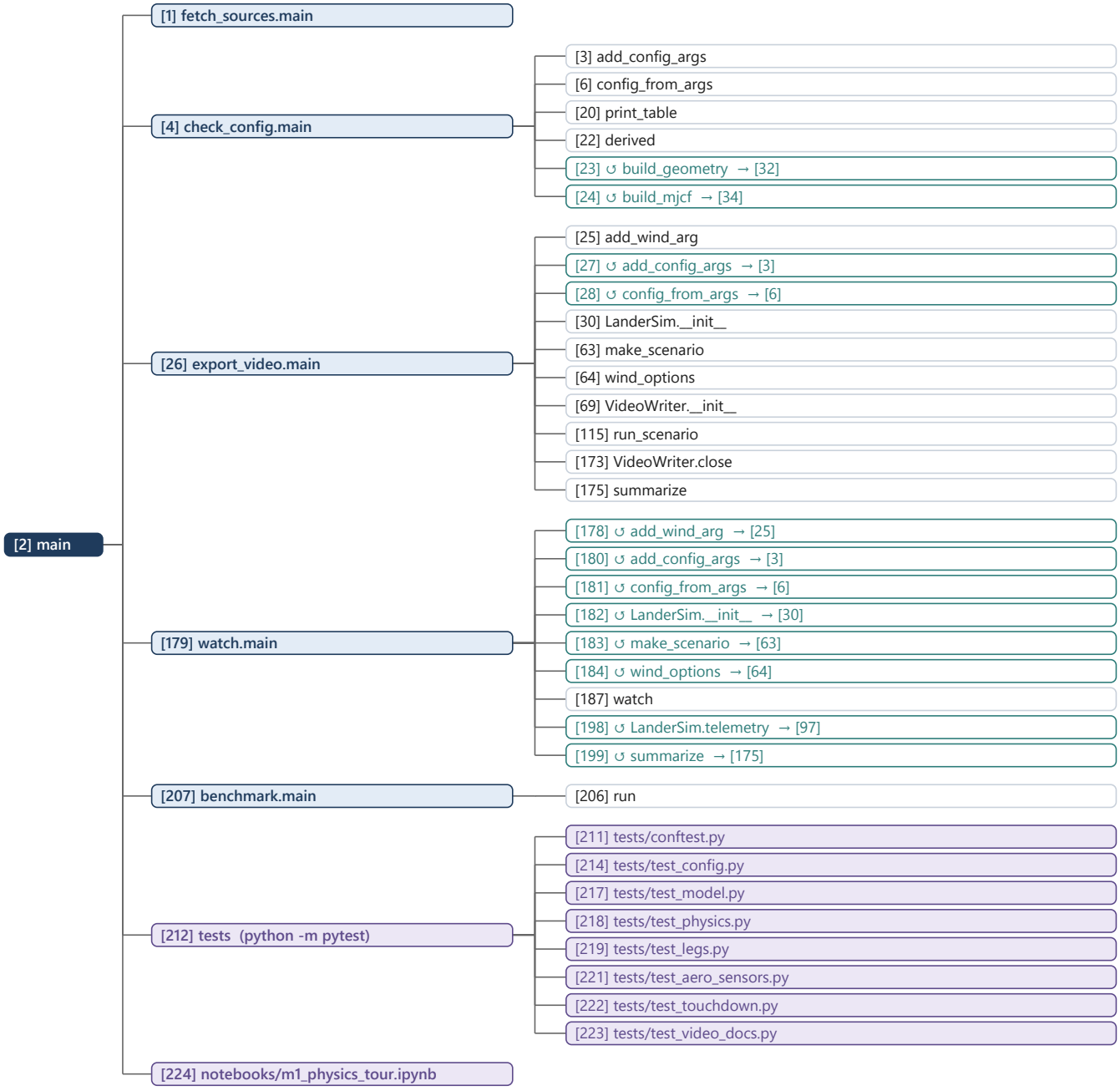


Figure 4 — main(), the entry points and what each calls first.

The complete call tree

```
[2] main
├─ [1] fetch_sources.main                                  scripts/fetch_sources.py:22
├─ [4] check_config.main                                   scripts/check_config.py:42
│  ├─ [3] add_config_args                                  scripts/common.py:10
│  ├─ [6] config_from_args                                 scripts/common.py:17
│  │  ├─ [5] parse_set_args                                lander/config.py:357
│  │  └─ [9] load_config                                   lander/config.py:335
│  │     ├─ [8] parse_env_file                             lander/config.py:235
│  │     │  └─ [7] parse_value                             lander/config.py:219
│  │     ├─ [11] _override                                 lander/config.py:269
│  │     │  └─ [10] ↺ parse_value → [7]
│  │     └─ [13] build_config                             lander/config.py:306
│  │        ├─ [12] _group_of                              lander/config.py:284
│  │        ├─ [14] _coerce                                lander/config.py:291
│  │        └─ [17] validate                               lander/config.py:372
│  │           ├─ [16] SimCfg.substeps                     lander/config.py:143
│  │           │  └─ [15] SimCfg.control_dt_s             lander/config.py:139
│  │           └─ [18] ↺ SimCfg.control_dt_s → [15]
│  ├─ [20] print_table                                     scripts/check_config.py:27
│  │  └─ [19] format_value                                 scripts/check_config.py:21
│  ├─ [22] derived                                         lander/config.py:405
│  │  └─ [21] ↺ SimCfg.substeps → [16]
│  ├─ [23] ↺ build_geometry → [32]
│  └─ [24] ↺ build_mjcf → [34]
├─ [26] export_video.main                                  scripts/export_video.py:19
│  ├─ [25] add_wind_arg                                    scripts/common.py:22
│  ├─ [27] ↺ add_config_args → [3]
│  ├─ [28] ↺ config_from_args → [6]
│  ├─ [30] LanderSim.__init__                              lander/sim.py:192
│  │  ├─ [29] ↺ load_config → [9]
│  │  ├─ [32] build_geometry                               lander/geometry.py:62
│  │  │  └─ [31] unit_azimuth                              lander/geometry.py:24
│  │  ├─ [41] build_model                                  lander/mjcf.py:207
│  │  │  ├─ [34] build_mjcf                                lander/mjcf.py:80
│  │  │  │  ├─ [33] MassModel.__init__                     lander/vehicle.py:40
│  │  │  │  ├─ [35] MassModel.total                        lander/vehicle.py:46
│  │  │  │  ├─ [37] hexagon_vertices                       lander/geometry.py:49
│  │  │  │  │  └─ [36] ↺ unit_azimuth → [31]
│  │  │  │  ├─ [38] _v                                     lander/mjcf.py:51
│  │  │  │  ├─ [39] _zquat                                 lander/mjcf.py:58
│  │  │  │  └─ [40] ↺ unit_azimuth → [31]
│  │  │  ├─ [42] mujoco.MjModel.from_xml_string(xml)
│  │  │  ├─ [43] build_model.gid                           lander/mjcf.py:212
│  │  │  ├─ [44] build_model.sensor_adr                    lander/mjcf.py:215
│  │  │  └─ [45] mujoco.mj_name2id(model, type, name)
│  │  ├─ [46] mujoco.MjData(model)
│  │  ├─ [48] Vehicle.__post_init__                        lander/vehicle.py:120
│  │  │  ├─ [47] EngineBank.__init__                       lander/vehicle.py:54
│  │  │  └─ [49] ↺ MassModel.__init__ → [33]
│  │  ├─ [51] SensorSuite.__init__                         lander/sensors.py:46
│  │  │  └─ [50] SensorSuite.reset                         lander/sensors.py:52
│  │  ├─ [52] TouchdownMonitor.__init__                    lander/touchdown.py:80
│  │  ├─ [53] PlaneTerrain.__init__                        lander/sim.py:165
│  │  ├─ [54] drag_areas                                   lander/aero.py:24
│  │  └─ [55] ↺ LanderSim.reset → [72]
│  ├─ [63] make_scenario                                   lander/scenarios.py:54
│  │  └─ [57] LanderSim.state_above_ground                 lander/sim.py:239
│  │     ├─ [56] terrain_normal                            lander/mjcf.py:62
│  │     ├─ [59] quat_from_tilt                            lander/sim.py:60
│  │     │  ├─ [58] quat_from_axis_angle                   lander/sim.py:54
│  │     │  └─ [60] quat_mul                              lander/sim.py:47
│  │     ├─ [61] quat_to_mat                               lander/sim.py:38
│  │     └─ [62] plane_height                              lander/sim.py:73
│  ├─ [64] wind_options                                    scripts/common.py:29
│  ├─ [69] VideoWriter.__init__                            lander/video.py:74
│  │  ├─ [66] FrameRenderer.__init__                       lander/video.py:54
│  │  │  ├─ [65] mujoco.Renderer(model, height, width)
│  │  │  └─ [68] named_camera                              lander/visuals.py:87
│  │  │     └─ [67] tracking_camera                        lander/visuals.py:78
│  │  └─ [70] imageio.get_writer(path, fps, codec='libx264')
│  ├─ [115] run_scenario                                   lander/scenarios.py:76
│  │  ├─ [72] LanderSim.reset                              lander/sim.py:262
│  │  │  ├─ [71] ResetOptions.from_any                     lander/sim.py:107
│  │  │  ├─ [73] mujoco.mj_resetData(model, data)
│  │  │  ├─ [75] set_terrain                               lander/mjcf.py:233
│  │  │  │  ├─ [74] ↺ terrain_normal → [56]
```

```
        └ [76] quat_z_to                                                lander/mjcf.py:68
    ├ [77] ↺ LanderSim.state_above_ground → [57]
    ├ [79] LanderSim._apply_mass                                        lander/sim.py:223
    │  └ [78] ↺ MassModel.total → [35]
    ├ [80] ↺ quat_to_mat → [61]
    ├ [81] ↺ cross3 → [98]
    ├ [83] LanderSim.hover_trim                                         lander/sim.py:232
    │  ├ [82] ↺ MassModel.total → [35]
    │  ├ [84] Vehicle.trim_thrust                                       lander/vehicle.py:147
    │  └ [85] EngineBank.thrust_to_valve                                lander/vehicle.py:71
    ├ [88] Vehicle.reset                                                lander/vehicle.py:125
    │  └ [87] EngineBank.reset                                          lander/vehicle.py:74
    │     └ [86] EngineBank.valve_to_thrust                             lander/vehicle.py:68
    ├ [89] wind_vector                                                  lander/aero.py:30
    ├ [90] ↺ SensorSuite.reset → [50]
    ├ [91] TouchdownMonitor.reset                                       lander/touchdown.py:86
    ├ [92] LanderSim.engines_off                                        lander/sim.py:219
    ├ [94] TouchdownMonitor.note_engines                                lander/touchdown.py:91
    │  └ [93] TouchdownMonitor._judge                                   lander/touchdown.py:137
    ├ [95] mujoco.mj_forward(model, data)
    └ [105] LanderSim._result                                           lander/sim.py:418
       ├ [97] LanderSim.telemetry                                       lander/sim.py:391
       │  ├ [96] ↺ quat_to_mat → [61]
       │  ├ [98] cross3                                                 lander/geometry.py:29
       │  ├ [101] PlaneTerrain.height_above                             lander/sim.py:173
       │  │  └ [100] PlaneTerrain.height                                lander/sim.py:170
       │  │     └ [99] ↺ plane_height → [62]
       │  ├ [102] LanderSim.t                                           lander/sim.py:215
       │  ├ [103] tilt_of                                               lander/sim.py:68
       │  └ [104] ↺ LanderSim.engines_off → [92]
       └ [107] LanderSim.read_sensors                                   lander/sim.py:409
          ├ [106] ↺ quat_to_mat → [61]
          └ [109] SensorSuite.sample                                    lander/sensors.py:60
             ├ [108] ↺ cross3 → [98]
             ├ [111] SensorSuite._read_altimeter                        lander/sensors.py:71
             │  └ [110] PlaneTerrain.nearest                            lander/sim.py:177
             └ [114] SensorSuite._read_doppler                          lander/sensors.py:80
                └ [113] PlaneTerrain.raycast                            lander/sim.py:181
                   └ [112] mujoco.mj_ray(model, data, pnt, vec, geomgroup, …)
 ├ [140] VideoWriter.add                                                lander/video.py:83
    ├ [117] FrameRenderer.render                                        lander/video.py:59
    │  ├ [116] Renderer.update_scene(data, camera)
    │  ├ [121] decorate                                                 lander/visuals.py:67
    │  │  ├ [120] add_plumes                                            lander/visuals.py:40
    │  │  │  └ [119] add_geom                                           lander/visuals.py:28
    │  │  │     └ [118] mujoco.mjv_initGeom(geom, type, size, pos, mat, rgba)
    │  │  ├ [122] ↺ LanderSim.telemetry → [97]
    │  │  └ [124] add_arrow                                             lander/visuals.py:59
    │  │     ├ [123] _frame_with_z                                      lander/visuals.py:19
    │  │     └ [125] ↺ add_geom → [119]
    │  ├ [126] Renderer.render()
    │  ├ [128] hud_lines                                                lander/hud.py:28
    │  │  ├ [127] SensorReadings.altitude_valid                         lander/sensors.py:37
    │  │  ├ [129] wind_heading_deg                                      lander/visuals.py:100
    │  │  ├ [130] Telemetry.vz                                          lander/sim.py:141
    │  │  ├ [131] Telemetry.vh                                          lander/sim.py:145
    │  │  ├ [132] Telemetry.spin_dps                                    lander/sim.py:149
    │  │  └ [134] status_text                                           lander/hud.py:10
    │  │     ├ [133] ↺ TouchdownReport.done → [163]
    │  │     ├ [136] TouchdownReport.success                            lander/touchdown.py:70
    │  │     │  └ [135] ↺ TouchdownReport.done → [163]
    │  │     └ [137] TouchdownReport.failures                           lander/touchdown.py:73
    │  └ [139] overlay_hud                                              lander/video.py:27
    │     └ [138] _font                                                 lander/video.py:20
    └ [141] writer.append_data(frame)
 ├ [143] hover_then_cut                                                 lander/scenarios.py:25
    └ [142] ↺ LanderSim.hover_trim → [83]
 ├ [145] hover                                                          lander/scenarios.py:20
    └ [144] ↺ LanderSim.hover_trim → [83]
 ├ [149] LanderSim.step                                                 lander/sim.py:309
    ├ [148] Vehicle.command                                             lander/vehicle.py:131
    │  └ [147] EngineBank.command                                       lander/vehicle.py:85
    │     └ [146] ↺ EngineBank.valve_to_thrust → [86]
    ├ [151] LanderSim._physics_step                                     lander/sim.py:323
    │  ├ [150] ↺ quat_to_mat → [61]
    │  ├ [152] ↺ cross3 → [98]
    │  ├ [154] Vehicle.step                                             lander/vehicle.py:134
    │  │  └ [153] EngineBank.step                                       lander/vehicle.py:90
```

```
        ├─ [155] drag_force_body                                  lander/aero.py:18
        ├─ [156] ↺ LanderSim._apply_mass → [79]
        ├─ [157] mujoco.mj_step(model, data)
        ├─ [158] LanderSim._contacts                              lander/sim.py:366
        ├─ [160] LanderSim._flag_events                           lander/sim.py:378
        │  └─ [159] ↺ LanderSim.engines_off → [92]
        ├─ [161] ↺ LanderSim.engines_off → [92]
        ├─ [162] ↺ TouchdownMonitor.note_engines → [94]
        ├─ [163] TouchdownReport.done                             lander/touchdown.py:65
        ├─ [164] stability_distance                               lander/touchdown.py:25
        ├─ [165] ↺ tilt_of → [103]
        └─ [167] TouchdownMonitor.update                          lander/touchdown.py:98
           ├─ [166] ↺ TouchdownMonitor._judge → [93]
           └─ [168] ↺ TouchdownReport.done → [163]
     ├─ [169] mujoco.mj_forward(model, data)
     └─ [170] ↺ LanderSim._result → [105]
  └─ [171] ↺ TouchdownReport.done → [163]
├─ [173] VideoWriter.close                                        lander/video.py:87
  └─ [172] FrameRenderer.close                                    lander/video.py:67
└─ [175] summarize                                                scripts/common.py:40
   ├─ [174] ↺ TouchdownReport.success → [136]
   ├─ [176] ↺ TouchdownReport.done → [163]
   └─ [177] ↺ TouchdownReport.failures → [137]
├─ [179] watch.main                                              scripts/watch.py:21
  ├─ [178] ↺ add_wind_arg → [25]
  ├─ [180] ↺ add_config_args → [3]
  ├─ [181] ↺ config_from_args → [6]
  ├─ [182] ↺ LanderSim.__init__ → [30]
  ├─ [183] ↺ make_scenario → [63]
  ├─ [184] ↺ wind_options → [64]
  ├─ [187] watch                                                 lander/viewer.py:22
  │  ├─ [186] reset_fn (in watch.main)                           scripts/watch.py:38
  │  │  └─ [185] ↺ LanderSim.reset → [72]
  │  ├─ [188] mujoco.viewer.launch_passive(model, data, key_callback, …)
  │  ├─ [189] on_key (in watch)                                  lander/viewer.py:32
  │  ├─ [190] ↺ tracking_camera → [67]
  │  ├─ [192] policy (in watch.main)                             scripts/watch.py:42
  │  │  └─ [191] ↺ hover_then_cut → [143]
  │  ├─ [193] ↺ LanderSim.step → [149]
  │  ├─ [194] ↺ decorate → [121]
  │  ├─ [196] hud_columns                                        lander/hud.py:52
  │  │  └─ [195] ↺ hud_lines → [128]
  │  └─ [197] handle.set_texts(…) / handle.sync()
  ├─ [198] ↺ LanderSim.telemetry → [97]
  └─ [199] ↺ summarize → [175]
├─ [207] benchmark.main                                          scripts/benchmark.py:44
  └─ [206] run                                                   scripts/benchmark.py:34
     └─ [201] _worker                                            scripts/benchmark.py:17
        ├─ [200] ↺ LanderSim.__init__ → [30]
        ├─ [202] ↺ make_scenario → [63]
        ├─ [203] ↺ LanderSim.reset → [72]
        ├─ [204] ↺ hover → [145]
        └─ [205] ↺ LanderSim.step → [149]
├─ [212] tests (python -m pytest)
  ├─ [211] tests/conftest.py                                     tests/conftest.py
  │  └─ [209] Config.with_overrides                              lander/config.py:205
  │     ├─ [208] ↺ _override → [11]
  │     └─ [210] ↺ build_config → [13]
  ├─ [214] tests/test_config.py                                  tests/test_config.py
  │  ├─ [213] Config.param                                       lander/config.py:182
  │  ├─ [215] Config.overridden                                  lander/config.py:192
  │  └─ [216] Config.to_log_dict                                 lander/config.py:196
  ├─ [217] tests/test_model.py                                   tests/test_model.py
  ├─ [218] tests/test_physics.py                                 tests/test_physics.py
  ├─ [219] tests/test_legs.py                                    tests/test_legs.py
  ├─ [221] tests/test_aero_sensors.py                            tests/test_aero_sensors.py
  │  └─ [220] SensorReadings.doppler_valid                       lander/sensors.py:41
  ├─ [222] tests/test_touchdown.py                               tests/test_touchdown.py
  └─ [223] tests/test_video_docs.py                              tests/test_video_docs.py
└─ [224] notebooks/m1_physics_tour.ipynb
```

6 Every node, in-order (LVR)

Nodes [1] to [224] in in-order. Full entries for functions, compact ones for references and external calls.

[1] fetch_sources.main(argv)

function · scripts/fetch_sources.py:22

Parent [2] main

Downloads the NASA source PDFs S3 (RP-1027), S4 (TN D-7959) and S5 (Pohlen et al.) into docs/sources/ with urllib, skipping files that already exist unless --force. The .env citations point to these.

Receives	argv → --dest (default docs/sources), --force (from [2])
Returns	None — three PDF files and their sizes printed

[2] main

program entry

L [1] fetch_sources.main · R [4] check_config.main, [26] export_video.main, [179] watch.main, [207] benchmark.main, [212] tests (python -m pytest), [224] notebooks/m1_physics_tour.ipynb

The program entry. Every command is `python -m scripts.<name> [options]`, and Python runs that script's `main()`. This root stands for that dispatch, so the whole project forms one tree. Its children go from the simplest command to the most complex, followed by the two other ways the library is driven: the test suite and the notebook.

Receives	command line: which script, plus its options (argv)
Returns	exit status; the command's effects (printed report, files in runs/ or docs/, a viewer window)
Calls	→ [1] **fetch_sources.main** — gets: argv → --dest (default docs/sources), --force; returns: None — three PDF files and their sizes printed → [4] **check_config.main** — gets: argv → --status, --grep, --dump-mjcf PATH, --set KEY=VALUE..., --env-file; returns: None — printed table; optional XML file → [26] **export_video.main** — gets: argv → --scenario, --seed, --out, --width, --height, --camera chase|ground|padcam, --no-hud, --wind, --set, --env-file; returns: Path of the MP4 (default runs/videos/<scenario>_seed<N>.mp4) → [179] **watch.main** — gets: argv → --scenario, --seed, --speed, --max-seconds, --no-hud, --wind, --set, --env-file; returns: None — a window, then a printed summary → [207] **benchmark.main** — gets: argv → --procs, --sim-seconds; returns: None — printed table → [212] **tests (python -m pytest)** — gets: pytest command line; returns: pass / fail report → [224] **notebooks/m1_physics_tour.ipynb** — gets: a Jupyter kernel running the .venv; returns: 6 figures and an embedded MP4

[3] add_config_args(parser)

function · scripts/common.py:10

Parent [4] check_config.main

Adds the two options every script shares: `--set KEY=VALUE` (repeatable) to override a fact for one run, and `--env-file` to load a different fact file.

Receives	parser: argparse.ArgumentParser (from [4])
Returns	None — the parser gains the two options

[4] check_config.main(argv)

function · scripts/check_config.py:42

Parent [2] main · L [3] add_config_args · R [6] config_from_args, [20] print_table, [22] derived, [23] build_geometry, [24] build_mjcf

Prints every fact from `.env` grouped by section with unit, status and source, then the derived sanity numbers and a count per status. With `--dump-mjcf` it also writes the generated MuJoCo XML.

Receives	argv → --status, --grep, --dump-mjcf PATH, --set KEY=VALUE..., --env-file (from [2])
Returns	None — printed table; optional XML file
Calls	→ [3] **add_config_args** — gets: parser: argparse.ArgumentParser; returns: None — the parser gains the two options → [6] **config_from_args** — gets: args: argparse.Namespace — `args.set` (list of 'KEY=VALUE'), `args.env_file` (path or None); returns: Config → [20] **print_table** — gets: cfg; status filter or None; grep substring or None; returns: None (stdout) → [22] **derived** — gets: cfg: Config; returns: dict name → (value, unit, formula) → [23] ↺ **build_geometry** — gets: cfg (only with --dump-mjcf); returns: LanderGeometry → [24] ↺ **build_mjcf** — gets: cfg, geom; returns: XML text, written to PATH

[5] parse_set_args(items)

function · lander/config.py:357

Parent [6] config_from_args

Splits each 'KEY=VALUE' item at the first '='. An item without '=' raises `ConfigError`.

Receives	items: list[str], e.g. ['VIKING_ENGINE_T90_S=0.1'] (from [6])
Returns	dict[str, str] — {key: raw value text}

[6] config_from_args(args)

function · scripts/common.py:17

Parent [4] check_config.main · L [5] parse_set_args · R [9] load_config

Turns the parsed command line into a validated `Config`: the `--set` items become an overrides dict (origin `cli`) and the fact file is loaded with them.

Receives	args: argparse.Namespace — `args.set` (list of 'KEY=VALUE'), `args.env_file` (path or None) (from [4])
Returns	Config
Calls	→ [5] **parse_set_args** — gets: items: list[str], e.g. ['VIKING_ENGINE_T90_S=0.1']; returns: dict[str, str] — {key: raw value text} → [9] **load_config** — gets: env_path (default ./.env); overrides: dict key → raw value; environ (default os.environ); origin label for the explicit overrides; returns: Config — every value plus its `Param` record (unit, status, source, note, origin)

[7] parse_value(text, key)

function · lander/config.py:219

Parent [8] parse_env_file

Converts the text of one value: an integer literal stays `int`, any other number becomes `float`, and comma-separated numbers become a tuple of floats.

Receives	text: str ('3', '0.5', '0.835,-0.835,0'); key (only for error messages) (from [8])
Returns	int | float | tuple[float, ...]

[8] parse_env_file(path)

function · lander/config.py:235

Parent [9] load_config · L [7] parse_value

Reads `.env` line by line. A fact line must be `KEY=value # unit | status | source | note`. Rejects unparsable lines, duplicate keys, a status other than published / derived / assumed, an empty source, and '-' (no source) unless the status is assumed. Comment lines, including the parked facts at the end of the file, are skipped.

Receives	path: Path of the fact file (from [9])
Returns	dict[str, Param] — origin '.env'
Calls	→ [7] **parse_value** — gets: text: str ('3', '0.5', '0.835,-0.835,0'); key (only for error messages); returns: int | float | tuple[float, ...]

[9] load_config(env_path, overrides, environ, origin)

function · lander/config.py:335

Parent [6] config_from_args · L [8] parse_env_file · R [11] _override, [13] build_config

The one way to obtain a `Config`. Reads the fact file, applies environment overrides `LANDER_<KEY>`, then explicit overrides, and builds the typed configuration. An unknown `LANDER_` variable or `--set` key is an error, so a typo can never be silently ignored.

Receives	env_path (default ./.env); overrides: dict key → raw value; environ (default os.environ); origin label for the explicit overrides (from [6])
Returns	Config — every value plus its `Param` record (unit, status, source, note, origin)
Calls	→ [8] **parse_env_file** — gets: path: Path of the fact file; returns: dict[str, Param] — origin '.env' → [11] **_override** — gets: old: Param; raw: str or number; origin: 'env:LANDER_<KEY>' | 'cli' | 'code' | 'test'; returns: Param with status 'override' → [13] **build_config** — gets: params: dict[str, Param]; env_path: str; returns: Config (frozen)

[10] ↺ **parse_value** reference: explained at [7]. Called by [11] _override — gets: raw text of the override; returns: the parsed number(s).

[11] _override(old, raw, origin)

function · lander/config.py:269

Parent [9] load_config · L [10] parse_value

Replaces one fact's value. The new `Param` keeps the unit, gets status `override`, records its origin and notes the old value, status and source. A list must keep its length.

Receives	old: Param; raw: str or number; origin: 'env:LANDER_<KEY>' | 'cli' | 'code' | 'test' (from [9])
Returns	Param with status 'override'
Calls	→ [10] ↺ **parse_value** — gets: raw text of the override; returns: the parsed number(s)

[12] _group_of(key)

function · lander/config.py:284

Parent [13] build_config

Finds the parameter group of a key from the ordered prefix table `GROUPS`.

Receives	key: str (from [13])
Returns	(prefix, group name, dataclass) or None

[13] build_config(params, env_path)

function · lander/config.py:306

Parent [9] load_config · L [12] _group_of · R [14] _coerce, [17] validate

Sorts the facts into the 11 group dataclasses by key prefix (the first matching prefix wins, so `MARS_WIND_` beats `MARS_`), checks that each group has exactly the keys its dataclass declares, converts every value to the declared type, then validates the whole configuration.

Receives	params: dict[str, Param]; env_path: str (from [9])
Returns	Config (frozen)
Calls	→ [12] **_group_of** — gets: key: str; returns: (prefix, group name, dataclass) or None → [14] **_coerce** — gets: key; value: int | float | tuple; typ: int | float | tuple[float, ...]; returns: the value in the declared type (`ConfigError` otherwise) → [17] **validate** — gets: cfg: Config; returns: None — raises `ConfigError` listing every problem

[14] _coerce(key, value, typ)

function · lander/config.py:291

Parent [13] build_config

Converts a parsed value to the field's declared type: `int` must be whole, `float` must be a single number, tuple fields accept one or more numbers.

Receives	key; value: int | float | tuple; typ: int | float | tuple[float, ...] (from [13])
Returns	the value in the declared type (`ConfigError` otherwise)

[15] SimCfg.control_dt_s

property · lander/config.py:139

Parent [16] SimCfg.substeps

Control period = 1 / `SIM_CONTROL_HZ` = 0.04 s: how long each command is held.

Receives	(self) `control_hz` (from [16])
Returns	float (s)

[16] SimCfg.substeps

property · lander/config.py:143

Parent [17] validate · L [15] SimCfg.control_dt_s

Physics steps per control step: round(control period / physics step) = 0.04 / 0.002 = 20.

Receives	(self) `physics_dt_s`, `control_hz` (from [17])
Returns	int
Calls	→ [15] **SimCfg.control_dt_s** — gets: (self) `control_hz`; returns: float (s)

[17] validate(cfg)

function · lander/config.py:372

Parent [13] build_config · L [16] SimCfg.substeps · R [18] SimCfg.control_dt_s

Cross-field sanity checks, all reported at once: one x and y for each of the 3 engines (D4); 0 < T_min < T_max; positive t90 and Isp; positive dry mass and non-negative start propellant; positive inertias obeying the triangle inequality; 3 leg azimuths and a positive footpad radius; a storm range 'low, high'; a positive radar period; a whole number of physics steps per control step; contact time constant ≥ 2 physics steps; a non-negative noise scale.

Receives	cfg: Config (from [13])
Returns	None — raises `ConfigError` listing every problem
Calls	→ [16] **SimCfg.substeps** — gets: (self) `physics_dt_s`, `control_hz`; returns: int → [18] ↺ **SimCfg.control_dt_s** — gets: —; returns: 0.04 s

[18] ↺ **SimCfg.control_dt_s** reference: explained at [15]. Called by [17] validate — gets: —; returns: 0.04 s.

[19] format_value(value)

function · scripts/check_config.py:21

Parent [20] print_table

Compact text for a number or a list.

Receives	value: int | float | tuple (from [20])
Returns	str, e.g. '0.835,-0.835,0' or '1.8'

[20] print_table(cfg, status, grep)

function · scripts/check_config.py:27

Parent [4] check_config.main · L [19] format_value

Prints the facts section by section ([engine] VIKING_ENGINE_*, ...): key, value, unit, status, source and note. Keys of a more specific prefix are kept out of the general section (MARS_WIND_* is not repeated under MARS_*).

Receives	cfg; status filter or None; grep substring or None (from [4])
Returns	None (stdout)
Calls	→ [19] **format_value** — gets: value: int | float | tuple; returns: str, e.g. '0.835,-0.835,0' or '1.8'

[21] ↺ **SimCfg.substeps** reference: explained at [16]. Called by [22] derived — gets: —; returns: 20.

[22] derived(cfg)

function · lander/config.py:405

Parent [4] check_config.main · L [21] SimCfg.substeps

Computes the sanity numbers from the facts: start mass and weight, propellant flow at maximum and minimum thrust, thrust-to-weight (maximum and the floor while lit), hover valve setting, hover flow and hover time on the start tank, the engine time constant, the push of the strongest storm, the CG height ratio and the physics substeps.

Receives	cfg: Config (from [4])
Returns	dict name → (value, unit, formula)
Calls	→ [21] ↺ **SimCfg.substeps** — gets: —; returns: 20

[23] ↺ **build_geometry** reference: explained at [32]. Called by [4] check_config.main — gets: cfg (only with --dump-mjcf); returns: LanderGeometry.

[24] ↺ **build_mjcf** reference: explained at [34]. Called by [4] check_config.main — gets: cfg, geom; returns: XML text, written to PATH.

[25] add_wind_arg(parser)

function · scripts/common.py:22

Parent [26] export_video.main

Adds `--wind SPEED[,DIRECTION]`: a steady wind for the episode (calm by default; Mars dust storms are 17–30 m/s).

Receives	parser (from [26])
Returns	None — the parser gains the option

[26] export_video.main(argv)

function · scripts/export_video.py:19

Parent [2] main · L [25] add_wind_arg · R [27] add_config_args, [28] config_from_args, [30] LanderSim.__init__, [63] make_scenario, [64] wind_options, [69] VideoWriter.__init__, [115] run_scenario, [173] VideoWriter.close, [175] summarize

Records a scenario to MP4 without opening a window: builds the simulator, makes the scenario, adds the `--wind` option, runs it while rendering one frame per control step (plumes + HUD), closes the file and prints the landing summary.

Receives	argv → --scenario, --seed, --out, --width, --height, --camera chase|ground|padcam, --no-hud, --wind, --set, --env-file (from [2])
Returns	Path of the MP4 (default runs/videos/<scenario>_seed<N>.mp4)
Calls	→ [25] **add_wind_arg** — gets: parser; returns: None — the parser gains the option → [27] ↺ **add_config_args** — gets: the script's parser; returns: None → [28] ↺ **config_from_args** — gets: parsed args; returns: Config → [30] **LanderSim.__init__** — gets: cfg: Config (None → `load_config()`, as the benchmark does); seed: int; returns: LanderSim (cfg, geom, model, idx, data, vehicle, sensors, monitor, terrain, dt, substeps, control_dt) → [63] **make_scenario** — gets: name; sim; seed; returns: Scenario (initial_state, options dict, duration, policy, seed) → [64] **wind_options** — gets: args; returns: dict of ResetOptions fields → [69] **VideoWriter.__init__** — gets: sim, path, width, height, camera name, hud flag, fps; returns: VideoWriter (frames, writer, count = 0) → [115] **run_scenario** — gets: sim, scenario, on_step(sim, StepResult) or None, settle_extra_s; returns: list[StepResult] — one per control step, including the reset → [173] **VideoWriter.close** — gets: —; returns: None → [175] **summarize** — gets: report: TouchdownReport, telemetry: Telemetry; returns: str

[27] ↺ **add_config_args** reference: explained at [3]. Called by [26] export_video.main — gets: the script's parser; returns: None.

[28] ↺ **config_from_args** reference: explained at [6]. Called by [26] export_video.main — gets: parsed args; returns: Config.

[29] ↺ **load_config** reference: explained at [9]. Called by [30] LanderSim.__init__ — gets: — (only when cfg is None); returns: Config.

[30] LanderSim.__init__(cfg, seed)

method · lander/sim.py:192

Parent [26] export_video.main · L [29] load_config · R [32] build_geometry, [41] build_model, [46] mujoco.MjData(model), [48] Vehicle.__post_init__, [51] SensorSuite.__init__, [52] TouchdownMonitor.__init__, [53] PlaneTerrain.__init__, [54] drag_areas, [55] LanderSim.reset

Builds a simulator once: geometry from the facts, the MuJoCo model compiled from generated XML, the MuJoCo state (`MjData`), and the backend-agnostic parts (vehicle, sensors, touchdown monitor, ground queries). Caches step sizes, drag areas, air density, the CG and inertia, then resets once so the object is ready to use.

Receives cfg: Config (None → `load_config()`, as the benchmark does); seed: int (from [26])

Returns LanderSim (cfg, geom, model, idx, data, vehicle, sensors, monitor, terrain, dt, substeps, control_dt)

Calls
→ [29] ↺ **load_config** — gets: — (only when cfg is None); returns: Config
→ [32] **build_geometry** — gets: cfg: Config (groups leg, engine, mass.cg_z_m, radar); returns: LanderGeometry (footpads, footpad_radius, leg azimuths, cg, engine_pos, radar_pos, doppler_beams)
→ [41] **build_model** — gets: cfg, geom; returns: (mujoco.MjModel, ModelIndex)
→ [46] **mujoco.MjData(model)** — gets: model; returns: MjData
→ [48] **Vehicle.__post_init__** — gets: cfg, geom (dataclass fields); returns: Vehicle (engines, mass, propellant = 0, empty = False)
→ [51] **SensorSuite.__init__** — gets: cfg, geom; returns: SensorSuite
→ [52] **TouchdownMonitor.__init__** — gets: cfg; returns: TouchdownMonitor
→ [53] **PlaneTerrain.__init__** — gets: sim; returns: PlaneTerrain
→ [54] **drag_areas** — gets: cfg.aero; returns: ndarray (3,) m^2
→ [55] ↺ **LanderSim.reset** — gets: seed; returns: first StepResult (discarded)

[31] unit_azimuth(az_rad)

function · lander/geometry.py:24

Parent [32] build_geometry

Horizontal unit vector (cos az, sin az, 0).

Receives az_rad: float (from [32])

Returns ndarray (3,)

[32] build_geometry(cfg)

function · lander/geometry.py:62

Parent [30] LanderSim.__init__ · L [31] unit_azimuth

Turns the geometry facts into vectors in the lander frame (TN D-7959 axes: origin on the centre line at the footpad plane, +Z up, leg 1 on +Y). The legs are rigid (D12), so everything is fixed: the three footpad contact-sphere centres, the CG on the centre line (D15), the engines on the body underside thrusting along +Z, both radar antennas at the centre of the underside, and the 4 Doppler beam directions (21.5° splay, 45° clock angle).

Receives cfg: Config (groups leg, engine, mass.cg_z_m, radar) (from [30])

Returns LanderGeometry (footpads, footpad_radius, leg azimuths, cg, engine_pos, radar_pos, doppler_beams)

Calls
→ [31] **unit_azimuth** — gets: az_rad: float; returns: ndarray (3,)

[33] MassModel.__init__(cfg, geom)

method · lander/vehicle.py:40

Parent [34] build_mjcf

Mass model of the simplified vehicle (D15): dry mass 573.1 kg, the CG fixed on the centre line at 0.79 m, the principal inertias fixed at NASA's touchdown values (S4 Fig 3).

Receives cfg.mass, geom.cg (from [34])

Returns MassModel (dry, cg, inertia)

[34] build_mjcf(cfg, geom)

function · lander/mjcf.py:80

Parent [41] build_model · L [33] MassModel.__init__ · R [35] MassModel.total, [37] hexagon_vertices, [38] _v, [39] _zquat, [40] unit_azimuth

Writes the MuJoCo model as text, so geometry always comes from `.env`: options (2 ms step, implicitfast integrator, Mars gravity, MuJoCo's fluid model off); visuals; assets (sky, ground texture, hexagonal body mesh); a mocap 'ground' body holding the terrain plane and the target marker; ONE free 'lander' body with the start mass, the CG and NASA's inertias, hull/radar-housing/engine/tank collision geoms, three footpad contact spheres, legs drawn as struts, sites and cameras; two sensors (IMU accelerometer and gyro at the CG). Sizes that only shape the drawing are constants in mjcf.py.

Receives cfg, geom (from [41])

Returns str — MJCF XML

Calls
→ [33] **MassModel.__init__** — gets: cfg.mass, geom.cg; returns: MassModel (dry, cg, inertia)
→ [35] **MassModel.total** — gets: propellant_kg; returns: float (kg)
→ [37] **hexagon_vertices** — gets: long_side, short_side (m); leg azimuths (rad); returns: ndarray (6, 2) — x, y in m
→ [38] **_v** — gets: *vals: numbers or arrays; returns: str
→ [39] **_zquat** — gets: az (rad); returns: str 'w x y z'
→ [40] ↺ **unit_azimuth** — gets: azimuths of the tanks, RTGs, antenna, boom, cameras and legs; returns: unit vectors

[35] MassModel.total(propellant_kg)

method · lander/vehicle.py:46

Parent [34] build_mjcf

Whole-vehicle mass with this much propellant left: dry + propellant (never below dry).

Receives	propellant_kg (from [34])
Returns	float (kg)

[36] ↺ **unit_azimuth** reference: explained at [31]. Called by [37] hexagon_vertices — gets: a leg azimuth and azimuth + 90°; returns: radial and tangential unit vectors.

[37] hexagon_vertices(long_side, short_side, leg_azimuths_rad)

function · lander/geometry.py:49

Parent [34] build_mjcf · L [36] unit_azimuth

Body outline from the S2 side lengths: an equilateral triangle of side L + 2S with its corners cut, short sides facing the legs. Returns the 6 corners counter-clockwise.

Receives	long_side, short_side (m); leg azimuths (rad) (from [34])
Returns	ndarray (6, 2) — x, y in m
Calls	→ [36] ↺ **unit_azimuth** — gets: a leg azimuth and azimuth + 90°; returns: radial and tangential unit vectors

[38] _v(*vals)

function · lander/mjcf.py:51

Parent [34] build_mjcf

Formats numbers and arrays as one space-separated string (6 significant digits) for XML attributes.

Receives	*vals: numbers or arrays (from [34])
Returns	str

[39] _zquat(az)

function · lander/mjcf.py:58

Parent [34] build_mjcf

Quaternion text for a rotation by az about +Z (target-ring markers, RTG covers).

Receives	az (rad) (from [34])
Returns	str 'w x y z'

[40] ↺ **unit_azimuth** reference: explained at [31]. Called by [34] build_mjcf — gets: azimuths of the tanks, RTGs, antenna, boom, cameras and legs; returns: unit vectors.

[41] build_model(cfg, geom)

function · lander/mjcf.py:207

Parent [30] LanderSim.__init__ · L [34] build_mjcf · R [42] mujoco.MjModel.from_xml_string(xml), [43] build_model.gid, [44] build_model.sensor_adr, [45] mujoco.mj_name2id(model, type, name)

Compiles the generated XML with MuJoCo and looks up by name every id and address the simulator touches each step: lander body, footpad and body geoms, the terrain geom, the ground mocap index, engine sites, IMU sensor-data addresses.

Receives	cfg, geom (from [30])
Returns	(mujoco.MjModel, ModelIndex)
Calls	→ [34] **build_mjcf** — gets: cfg, geom; returns: str — MJCF XML → [42] **mujoco.MjModel.from_xml_string(xml)** — gets: xml: str; returns: MjModel → [43] **build_model.gid** — gets: name: str; returns: int geom id → [44] **build_model.sensor_adr** — gets: name: str; returns: int address → [45] **mujoco.mj_name2id(model, type, name)** — gets: model, object type, name; returns: int

[42] external `mujoco.MjModel.from_xml_string(xml)` — MuJoCo compiles the XML into an MjModel: every constant of the simulation (bodies, geoms, sites, sensors, options). Called by [41] build_model — gets: xml: str; returns: MjModel.

[43] build_model.gid(name)

nested function · lander/mjcf.py:212

Parent [41] build_model

Helper inside build_model: id of a named geom.

Receives	name: str (from [41])
Returns	int geom id

[44] build_model.sensor_adr(name)

nested function · lander/mjcf.py:215

Parent [41] build_model

Helper inside build_model: start address of a named sensor's values in `data.sensordata`.

Receives	name: str (from [41])
Returns	int address

[45] external `mujoco.mj_name2id(model, type, name)` — Integer id of a named body or site. Called by [41] build_model — gets: model, object type, name; returns: int.

[46] external `mujoco.MjData(model)` — Allocates the simulation state: qpos (7 = frame origin xyz and attitude quaternion), qvel (6), time, applied forces, contacts, sensor data. Called by [30] LanderSim.__init__ — gets: model; returns: MjData.

[47] EngineBank.__init__(cfg, geom)

method · lander/vehicle.py:54

Parent [48] Vehicle.__post_init__

Reads the engine facts once: thrust limits 276 / 2,667 N, $Isp \cdot g_0$, the first-order time constant τ = t90 / ln 10 = 23.0 ms (the same up and down), the misalignment angle and the engine positions.

Receives	cfg.engine, geom.engine_pos (from [48])
Returns	EngineBank (thrust = cmd = 0, lit, thrust directions +Z)

[48] Vehicle.__post_init__()

method · lander/vehicle.py:120

Parent [30] LanderSim.__init__ · L [47] EngineBank.__init__ · R [49] MassModel.__init__

Creates the vehicle layer's two parts after the dataclass fields are set (engine bank and mass model) and caches the engines' lever arms about the CG, which never change because the CG is fixed.

Receives	cfg, geom (dataclass fields) (from [30])
Returns	Vehicle (engines, mass, propellant = 0, empty = False)
Calls	→ [47] **EngineBank.__init__** — gets: cfg.engine, geom.engine_pos; returns: EngineBank (thrust = cmd = 0, lit, thrust directions +Z) → [49] ↺ **MassModel.__init__** — gets: cfg, geom; returns: MassModel

[49] ↺ **MassModel.__init__** reference: explained at [33]. Called by [48] Vehicle.__post_init__ — gets: cfg, geom; returns: MassModel.

[50] SensorSuite.reset(rng, noise_scale)

method · lander/sensors.py:52

Parent [51] SensorSuite.__init__

Takes this episode's noise generator and noise scale (Viking's radar errors × scale; 1 by default) and clears the radars' sample-and-hold so the next sample is taken at once.

Receives	rng; noise_scale or None (the .env value) (from [51])
Returns	None

[51] SensorSuite.__init__(cfg, geom)

method · lander/sensors.py:46

Parent [30] LanderSim.__init__ · L [50] SensorSuite.reset

Stores the Doppler beam directions and the noise scale, then resets.

Receives	cfg, geom (from [30])
Returns	SensorSuite
Calls	→ [50] **SensorSuite.reset** — gets: rng; noise_scale or None (the .env value); returns: None

[52] TouchdownMonitor.__init__(cfg)

method · lander/touchdown.py:80

Parent [30] LanderSim.__init__

Keeps the section 7 limits (`cfg.req`) and an empty `TouchdownReport`.

Receives	cfg (from [30])
Returns	TouchdownMonitor

[53] PlaneTerrain.__init__(sim)

method · lander/sim.py:165

Parent [30] LanderSim.__init__

Ground-query object for the sensor models: normal n = +Z until reset tilts it, and a back-reference to the simulator for MuJoCo ray casts.

Receives	sim (from [30])
Returns	PlaneTerrain

[54] drag_areas(cfg)

function · lander/aero.py:24

Parent [30] LanderSim.__init__

$C_d \cdot A$ for each body axis: lateral x, y = 2.01 m² (calibrated to S4's touchdown aero forces), axial z = 1.80 m².

Receives	cfg.aero (from [30])
Returns	ndarray (3,) m²

[55] ↺ **LanderSim.reset** reference: explained at [72]. Called by [30] LanderSim.__init__ — gets: seed; returns: first StepResult (discarded).

[56] terrain_normal(slope_deg, downhill_deg)

function · lander/mjcf.py:62

Parent [57] LanderSim.state_above_ground

Unit normal of a plane through the origin that falls toward `downhill_deg`: (sin s cos a, sin s sin a, cos s).

Receives	slope_deg, downhill_deg (from [57])
Returns	ndarray (3,)

[57] LanderSim.state_above_ground(pad_altitude, velocity, tilt_deg, tilt_azimuth_deg, heading_deg, omega, xy, propellant_kg, throttle, slope_deg, slope_direction_deg)

method · lander/sim.py:239

Parent [63] make_scenario · L [56] terrain_normal · R [59] quat_from_tilt, [61] quat_to_mat, [62] plane_height

Turns 'pad_altitude metres above the ground, moving at velocity, tilted like this' into the CG state the simulator needs. It builds the attitude, rotates the three footpads, and places the lander frame so the lowest footpad sphere is exactly pad_altitude above the (possibly sloped) ground below it.

Receives	pad_altitude (m); velocity (3, m/s world); tilt_deg; tilt_azimuth_deg; heading_deg; omega (3, rad/s body); xy; propellant_kg; throttle; slope_deg; slope_direction_deg (from [63])
Returns	InitialState (CG position and velocity in the world, quaternion, body rates, propellant, throttle)
Calls	→ [56] **terrain_normal** — gets: slope_deg, downhill_deg; returns: ndarray (3,) → [59] **quat_from_tilt** — gets: tilt_deg, tilt_azimuth_deg, heading_deg; returns: quaternion (w, x, y, z) → [61] **quat_to_mat** — gets: q: 4 numbers; returns: ndarray (3, 3) → [62] **plane_height** — gets: n (3,), x, y; returns: float (m)

[58] quat_from_axis_angle(axis, angle)

function · lander/sim.py:54

Parent [59] quat_from_tilt

Quaternion of a rotation by an angle about an axis.

Receives	axis (3,), angle (rad) (from [59])
Returns	quaternion

[59] quat_from_tilt(tilt_deg, tilt_azimuth_deg, heading_deg)

function · lander/sim.py:60

Parent [57] LanderSim.state_above_ground · L [58] quat_from_axis_angle · R [60] quat_mul

Attitude quaternion: rotate `heading` about +Z, then lean the thrust axis by `tilt` toward `tilt_azimuth`.

Receives	tilt_deg, tilt_azimuth_deg, heading_deg (from [57])
Returns	quaternion (w, x, y, z)
Calls	→ [58] **quat_from_axis_angle** — gets: axis (3,), angle (rad); returns: quaternion → [60] **quat_mul** — gets: a, b: quaternions; returns: quaternion

[60] quat_mul(a, b)

function · lander/sim.py:47

Parent [59] quat_from_tilt

Quaternion (Hamilton) product a·b: apply b, then a.

Receives	a, b: quaternions (from [59])
Returns	quaternion

[61] quat_to_mat(q)

function · lander/sim.py:38

Parent [57] LanderSim.state_above_ground

Rotation matrix body → world from a unit quaternion (w, x, y, z).

Receives	q: 4 numbers (from [57])
Returns	ndarray (3, 3)

[62] plane_height(n, x, y)

function · lander/sim.py:73

Parent [57] LanderSim.state_above_ground

Ground height z at (x, y) for a plane through the origin with normal n.

Receives	n (3,), x, y (from [57])
Returns	float (m)

[63] make_scenario(name, sim, seed)

function · lander/scenarios.py:54

Parent [26] export_video.main · L [57] LanderSim.state_above_ground

Builds one of the five scripted cases: drop (6 m up, 2.44 m/s down, 6 kg propellant), hard_drop (1 m, 4 m/s down + 0.6 m/s sideways), slope45 (2.44 m/s onto a 45° slope), hover (30 m up, start tank, never cuts) and freefall (100 m, engines off). All but hover cut the engines at first contact.

Receives	name; sim; seed (from [26])
Returns	Scenario (initial_state, options dict, duration, policy, seed)
Calls	→ [57] **LanderSim.state_above_ground** — gets: pad_altitude (m); velocity (3, m/s world); tilt_deg; tilt_azimuth_deg; heading_deg; omega (3, rad/s body); xy; propellant_kg; throttle; slope_deg; slope_direction_deg; returns: InitialState (CG position and velocity in the world, quaternion, body rates, propellant, throttle)

[64] wind_options(args)

function · scripts/common.py:29

Parent [26] export_video.main

Turns `--wind 25,90` into reset options {wind_speed_mps: 25, wind_direction_deg: 90}; empty when the flag is not given.

Receives	args (from [26])
Returns	dict of ResetOptions fields

[65] external `mujoco.Renderer(model, height, width)` — Offscreen OpenGL renderer with its own MjvScene. Called by [66] FrameRenderer.__init__ — gets: model, size; returns: Renderer.

[66] FrameRenderer.__init__(sim, width, height, camera, hud)

method · lander/video.py:54

Parent [69] VideoWriter.__init__ · L [65] mujoco.Renderer(model, height, width) · R [68] named_camera

Creates MuJoCo's offscreen renderer (an OpenGL context: hidden GLFW window on Windows, EGL on headless Linux) and the camera.

Receives	sim, width, height, camera, hud (from [69])
Returns	FrameRenderer
Calls	→ [65] **mujoco.Renderer(model, height, width)** — gets: model, size; returns: Renderer → [68] **named_camera** — gets: sim, name; returns: MjvCamera

[67] tracking_camera(sim, distance, azimuth, elevation)

function · lander/visuals.py:78

Parent [68] named_camera

Camera that follows the lander's centre of mass at a fixed, world-aligned distance, azimuth and elevation (10 m, 135°, −12°), so wind and drift directions read correctly.

Receives	sim, distance, azimuth, elevation (from [68])
Returns	MjvCamera

[68] named_camera(sim, name)

function · lander/visuals.py:87

Parent [66] FrameRenderer.__init__ · L [67] tracking_camera

'chase' → a camera that follows the lander; 'ground' or 'padcam' → the fixed cameras defined in the MJCF.

Receives	sim, name (from [66])
Returns	MjvCamera
Calls	→ [67] **tracking_camera** — gets: sim, distance, azimuth, elevation; returns: MjvCamera

[69] VideoWriter.__init__(sim, path, width, height, camera, hud, fps)

method · lander/video.py:74

Parent [26] export_video.main · L [66] FrameRenderer.__init__ · R [70] imageio.get_writer(path, fps, codec='libx264')

Opens an MP4 writer (libx264 through imageio-ffmpeg) at `SIM_CONTROL_HZ` frames per second, so one frame per control step plays back in real time, and creates the frame renderer.

Receives	sim, path, width, height, camera name, hud flag, fps (from [26])
Returns	VideoWriter (frames, writer, count = 0)
Calls	→ [66] **FrameRenderer.__init__** — gets: sim, width, height, camera, hud; returns: FrameRenderer → [70] **imageio.get_writer(path, fps, codec='libx264')** — gets: path, fps, codec, quality; returns: writer

[70] external `imageio.get_writer(path, fps, codec='libx264')` — ffmpeg-backed MP4 writer from imageio-ffmpeg. Called by [69] VideoWriter.__init__ — gets: path, fps, codec, quality; returns: writer.

[71] ResetOptions.from_any(options)

method · lander/sim.py:107

Parent [72] LanderSim.reset

Accepts None (all defaults), a `ResetOptions` or a dict of its fields; unknown keys raise an error listing the valid ones.

Receives	options (from [72])
Returns	ResetOptions

[72] LanderSim.reset(seed, initial_state, options)

method · lander/sim.py:262

Parent [115] run_scenario · L [71] ResetOptions.from_any · R [73] mujoco.mj_resetData(model, data), [75] set_terrain, [77] LanderSim.state_above_ground, [79] LanderSim._apply_mass, [80] quat_to_mat, [81] cross3, [83] LanderSim.hover_trim, [88] Vehicle.reset, [89] wind_vector, [90] SensorSuite.reset, [91] TouchdownMonitor.reset, [92] LanderSim.engines_off, [94] TouchdownMonitor.note_engines, [95] mujoco.mj_forward(model, data), [105] LanderSim._result

Starts an episode. Spawns two random streams (engine misalignment, radar noise) from one seed in a fixed order (D10); resets MuJoCo; tilts the ground and sets friction; sets the mass for the starting propellant; writes qpos and qvel from the CG state; picks the starting throttle (hover trim if none given); resets the vehicle, the episode's steady wind, the sensors and the touchdown monitor; zeroes the applied forces; recomputes MuJoCo; returns the first StepResult.

Receives	seed (int or None = keep the last); initial_state: InitialState (None = 30 m hover); options: ResetOptions, dict or None (from [115])
Returns	StepResult (telemetry at t = 0, sensor readings, empty report, events [])
State	writes data.qpos, data.qvel, the ground mocap pose, the model mass; stores options, wind, slope, friction
Calls	→ [71] **ResetOptions.from_any** — gets: options; returns: ResetOptions → [73] **mujoco.mj_resetData(model, data)** — gets: model, data; returns: None → [75] **set_terrain** — gets: model, data, idx, slope_deg, downhill_deg, friction; returns: ground normal (3,) → [77] ↺ **LanderSim.state_above_ground** — gets: 30 m (only when no initial_state is given); returns: InitialState → [79] **LanderSim._apply_mass** — gets: propellant (kg); returns: None → [80] ↺ **quat_to_mat** — gets: initial quaternion; returns: rotation R → [81] ↺ **cross3** — gets: ω_world × R·cg; returns: velocity offset between CG and frame origin → [83] **LanderSim.hover_trim** — gets: propellant (default: current); total_thrust (default: weight); returns: tuple of 3 valve settings → [88] **Vehicle.reset** — gets: rng, propellant_kg, throttle0 (3), engines_enabled, fault_scale (3); returns: None → [89] **wind_vector** — gets: speed_mps, direction_deg (reset options); returns: ndarray (3,) m/s, world frame → [90] ↺ **SensorSuite.reset** — gets: rng, sensor_noise_scale option; returns: None → [91] **TouchdownMonitor.reset** — gets: —; returns: None → [92] **LanderSim.engines_off** — gets: (self) — reads vehicle.engines.lit; returns: bool → [94] **TouchdownMonitor.note_engines** — gets: engines_off: bool; returns: None → [95] **mujoco.mj_forward(model, data)** — gets: model, data; returns: None → [105] **LanderSim._result** — gets: events: list[str]; returns: StepResult(telemetry, sensors, report, events)

[73] external `mujoco.mj_resetData(model, data)` — Back to the model's defaults: time 0, default qpos, zero velocities, default mocap poses. Called by [72] LanderSim.reset — gets: model, data; returns: None.

[74] ↺ **terrain_normal** reference: explained at [56]. Called by [75] set_terrain — gets: slope_deg, downhill_deg; returns: normal n.

[75] set_terrain(model, data, idx, slope_deg, downhill_deg, friction)

function · lander/mjcf.py:233

Parent [72] LanderSim.reset · L [74] terrain_normal · R [76] quat_z_to

Tilts the ground for this episode by setting the 'ground' mocap body's quaternion (the plane passes through the target at the origin) and sets the terrain friction (contact friction = the larger of the two geoms'; footpads use 0.01).

Receives	model, data, idx, slope_deg, downhill_deg, friction (from [72])
Returns	ground normal (3,)
Calls	→ [74] ↺ **terrain_normal** — gets: slope_deg, downhill_deg; returns: normal n → [76] **quat_z_to** — gets: n: (3,); returns: quaternion

[76] quat_z_to(n)

function · lander/mjcf.py:68

Parent [75] set_terrain

Quaternion that rotates +Z onto a unit vector.

Receives	n: (3,) (from [75])
Returns	quaternion

[77] ↺ **LanderSim.state_above_ground** reference: explained at [57]. Called by [72] LanderSim.reset — gets: 30 m (only when no initial_state is given); returns: InitialState.

[78] ↺ **MassModel.total** reference: explained at [35]. Called by [79] LanderSim._apply_mass — gets: propellant; returns: mass (kg).

[79] LanderSim._apply_mass(propellant)

method · lander/sim.py:223

Parent [72] LanderSim.reset · L [78] MassModel.total

Writes the lander's mass (dry + propellant) into the MuJoCo model; the CG and inertia never change (D15), so nothing else is touched.

Receives	propellant (kg) (from [72])
Returns	None
State	writes model.body_mass and body_subtreemass; sets _mass
Calls	→ [78] ↺ **MassModel.total** — gets: propellant; returns: mass (kg)

[80] ↺ **quat_to_mat** reference: explained at [61]. Called by [72] LanderSim.reset — gets: initial quaternion; returns: rotation R.

[81] ↺ **cross3** reference: explained at [98]. Called by [72] LanderSim.reset — gets: ω_world × R·cg; returns: velocity offset between CG and frame origin.

[82] ↺ **MassModel.total** reference: explained at [35]. Called by [83] LanderSim.hover_trim — gets: propellant; returns: mass for the weight.

[83] LanderSim.hover_trim(propellant, total_thrust)

method · lander/sim.py:232

Parent [72] LanderSim.reset · L [82] MassModel.total · R [84] Vehicle.trim_thrust, [85] EngineBank.thrust_to_valve

Valve settings that give a total thrust (default: the current weight on Mars) with zero pitch and yaw torque about the CG.

Receives	propellant (default: current); total_thrust (default: weight) (from [72])
Returns	tuple of 3 valve settings
Calls	→ [82] ↺ **MassModel.total** — gets: propellant; returns: mass for the weight → [84] **Vehicle.trim_thrust** — gets: total_thrust (N); returns: ndarray (3,) N → [85] **EngineBank.thrust_to_valve** — gets: thrust (N); returns: valve setting(s)

[84] Vehicle.trim_thrust(total_thrust)

method · lander/vehicle.py:147

Parent [83] LanderSim.hover_trim

Solves ΣT = F, ΣT·y = 0, ΣT·x = 0 for the three engine thrusts. Engine 3 sits a little further out (S4 Fig 6a), so it runs ~2.5 % lower than engines 1 and 2.

Receives	total_thrust (N) (from [83])
Returns	ndarray (3,) N

[85] EngineBank.thrust_to_valve(t)

method · lander/vehicle.py:71

Parent [83] LanderSim.hover_trim

Inverse of the valve law: u = (T − T_min) / (T_max − T_min), not clipped.

Receives	thrust (N) (from [83])
Returns	valve setting(s)

[86] EngineBank.valve_to_thrust(u)

method · lander/vehicle.py:68

Parent [87] EngineBank.reset

Viking valve law (S3 p96): T = T_min + clip(u, 0, 1)·(T_max − T_min). u = 0 gives 276 N, not off.

Receives	u: one or three valve settings (from [87])
Returns	thrust (N)

[87] EngineBank.reset(rng, throttle0, enabled, fault_scale)

method · lander/vehicle.py:74

Parent [88] Vehicle.reset · L [86] EngineBank.valve_to_thrust

Per episode: lit flag, per-engine fault scale, and each thrust direction leaned by the misalignment angle (0.044°) toward a random azimuth. The engines start at the commanded throttle with no transient.

Receives	rng, throttle0 (3), enabled, fault_scale (3) (from [88])
Returns	None
Calls	→ [86] **EngineBank.valve_to_thrust** — gets: u: one or three valve settings; returns: thrust (N)

[88] Vehicle.reset(rng, propellant_kg, throttle0, engines_enabled, fault_scale)

method · lander/vehicle.py:125

Parent [72] LanderSim.reset · L [87] EngineBank.reset

Sets the propellant, marks the tank empty if it is, and resets the engines (off when they are disabled or the tank is empty).

Receives	rng, propellant_kg, throttle0 (3), engines_enabled, fault_scale (3) (from [72])
Returns	None
Calls	→ [87] **EngineBank.reset** — gets: rng, throttle0 (3), enabled, fault_scale (3); returns: None

[89] wind_vector(speed_mps, direction_deg)

function · lander/aero.py:30

Parent [72] LanderSim.reset

The episode's steady wind: speed × (cos dir, sin dir, 0), blowing toward `dir`. Calm episodes pass 0; dust storms are 17–30 m/s.

Receives	speed_mps, direction_deg (reset options) (from [72])
Returns	ndarray (3,) m/s, world frame

[90] ↺ **SensorSuite.reset** reference: explained at [50]. Called by [72] LanderSim.reset — gets: rng, sensor_noise_scale option; returns: None.

[91] TouchdownMonitor.reset()

method · lander/touchdown.py:86

Parent [72] LanderSim.reset

Starts a fresh `TouchdownReport` and clears the settle timer.

Receives	— (from [72])
Returns	None

[92] LanderSim.engines_off

property · lander/sim.py:219

Parent [72] LanderSim.reset

True once the engines are shut down, by the engines-off command or an empty tank. Permanent: nothing relights them.

Receives	(self) — reads vehicle.engines.lit (from [72])
Returns	bool

[93] TouchdownMonitor._judge()

method · lander/touchdown.py:137

Parent [94] TouchdownMonitor.note_engines

Fills report.criteria per brief §7: vertical speed ≤ 3.34 m/s and horizontal ≤ 1.22 m/s at first contact (otherwise a leg broke), stable (once settled or tipped), no body strike, engines off, propellant left at touchdown. inside_pad_radius and tilt_within_limit stay None until O3 (M2).

Receives	— (from [94])
Returns	None

[94] TouchdownMonitor.note_engines(engines_off)

method · lander/touchdown.py:91

Parent [72] LanderSim.reset · L [93] TouchdownMonitor._judge

Records every physics step whether the engines are shut down, even after the landing is decided, and re-judges the criteria when that changes after touchdown: a landing only counts once the engines are off.

Receives	engines_off: bool (from [72])
Returns	None
Calls	→ [93] **TouchdownMonitor._judge** — gets: —; returns: None

[95] external `mujoco.mj_forward(model, data)` — Recomputes everything derived from the state (positions, contacts, sensor data) without advancing time. Called by [72] LanderSim.reset — gets: model, data; returns: None.

[96] ↺ **quat_to_mat** reference: explained at [61]. Called by [97] LanderSim.telemetry — gets: data.qpos[3:7]; returns: rotation R.

[97] LanderSim.telemetry()

method · lander/sim.py:391

Parent [105] LanderSim._result · L [96] quat_to_mat · R [98] cross3, [101] PlaneTerrain.height_above, [102] LanderSim.t, [103] tilt_of, [104] LanderSim.engines_off

Ground-truth snapshot from the MuJoCo state: CG position and velocity (frame origin plus the rotated CG offset), attitude, body rates, tilt, CG height and lowest-footpad height above the ground, propellant, mass, actual thrusts, last throttle command, whether the engines are off, the wind and the contacts.

Receives	— reads data.qpos, data.qvel, data.geom_xpos and the components' state (from [105])
Returns	Telemetry
Calls	→ [96] ↺ **quat_to_mat** — gets: data.qpos[3:7]; returns: rotation R → [98] **cross3** — gets: a, b: (3,); returns: ndarray (3,) → [101] **PlaneTerrain.height_above** — gets: p: (3,) world; returns: float (m) → [102] **LanderSim.t** — gets: (self); returns: float (s) → [103] **tilt_of** — gets: rot: (3, 3); returns: float (deg) → [104] ↺ **LanderSim.engines_off** — gets: —; returns: bool

[98] cross3(a, b)

function · lander/geometry.py:29

Parent [97] LanderSim.telemetry

Cross product of two 3-vectors written out with floats; np.cross was a bottleneck of the physics loop.

Receives	a, b: (3,) (from [97])
Returns	ndarray (3,)

[99] ↺ **plane_height** reference: explained at [62]. Called by [100] PlaneTerrain.height — gets: n, x, y; returns: z.

[100] PlaneTerrain.height(x, y)

method · lander/sim.py:170

Parent [101] PlaneTerrain.height_above · L [99] plane_height

Ground height at (x, y).

Receives	x, y (from [101])
Returns	z (m)
Calls	→ [99] ↺ **plane_height** — gets: n, x, y; returns: z

[101] PlaneTerrain.height_above(p)

method · lander/sim.py:173

Parent [97] LanderSim.telemetry · L [100] PlaneTerrain.height

Vertical height of a point above the ground directly below it.

Receives	p: (3,) world (from [97])
Returns	float (m)
Calls	→ [100] **PlaneTerrain.height** — gets: x, y; returns: z (m)

[102] LanderSim.t

property · lander/sim.py:215

Parent [97] LanderSim.telemetry

Simulation time = `data.time`.

Receives	(self) (from [97])
Returns	float (s)

[103] tilt_of(rot)

function · lander/sim.py:68

Parent [97] LanderSim.telemetry

Angle between the body +Z (thrust) axis and world +Z: acos(R[2, 2]).

Receives	rot: (3, 3) (from [97])
Returns	float (deg)

[104] ↺ **LanderSim.engines_off** reference: explained at [92]. Called by [97] LanderSim.telemetry — gets: —; returns: bool.

[105] LanderSim._result(events)

method · lander/sim.py:418

Parent [72] LanderSim.reset · L [97] LanderSim.telemetry · R [107] LanderSim.read_sensors

Packs a control step's outputs.

Receives	events: list[str] (from [72])
Returns	StepResult(telemetry, sensors, report, events)
Calls	→ [97] **LanderSim.telemetry** — gets: — reads data.qpos, data.qvel, data.geom_xpos and the components' state; returns: Telemetry → [107] **LanderSim.read_sensors** — gets: — reads data.sensordata, data.qpos, data.qvel; returns: SensorReadings

[106] ↺ **quat_to_mat** reference: explained at [61]. Called by [107] LanderSim.read_sensors — gets: data.qpos[3:7]; returns: rotation R.

[107] LanderSim.read_sensors()

method · lander/sim.py:409

Parent [105] LanderSim._result · L [106] quat_to_mat · R [109] SensorSuite.sample

Feeds the sensor models with the truth: specific force and body rates from MuJoCo's accelerometer and gyro at the IMU site (the CG), the frame pose and velocities, and the ground-query object.

Receives	— reads data.sensordata, data.qpos, data.qvel (from [105])
Returns	SensorReadings
Calls	→ [106] ↺ **quat_to_mat** — gets: data.qpos[3:7]; returns: rotation R → [109] **SensorSuite.sample** — gets: t, f_body (3), omega_body (3), origin (3), rot (3×3), v_origin (3), omega_world (3), terrain; returns: SensorReadings (accel, gyro, altitude, doppler_beam × 4, doppler_vel)

[108] ↺ **cross3** reference: explained at [98]. Called by [109] SensorSuite.sample — gets: ω × r (antenna offset from the frame origin); returns: rotation part of the antenna velocity.

[109] SensorSuite.sample(t, f_body, omega_body, origin, rot, v_origin, omega_world, terrain)

method · lander/sensors.py:60

Parent [107] LanderSim.read_sensors · L [108] cross3 · R [111] SensorSuite._read_altimeter, [114] SensorSuite._read_doppler

The IMU is ideal: it passes the true specific force and rates through. The radars are sampled only every 200 ms and hold their values in between (S3 p93): it computes the antenna position and velocity at the centre of the underside, then reads the altimeter and the Doppler radar.

Receives	t, f_body (3), omega_body (3), origin (3), rot (3×3), v_origin (3), omega_world (3), terrain (from [107])
Returns	SensorReadings (accel, gyro, altitude, doppler_beam × 4, doppler_vel)
Calls	→ [108] ↺ **cross3** — gets: ω × r (antenna offset from the frame origin); returns: rotation part of the antenna velocity → [111] **SensorSuite._read_altimeter** — gets: p_ant (3), rot, terrain; returns: float (m), NaN = invalid → [114] **SensorSuite._read_doppler** — gets: p_ant (3), v_ant (3), rot, terrain; returns: (beam speeds (4,), velocity (3,)) — NaN = no lock

[110] PlaneTerrain.nearest(p)

method · lander/sim.py:177

Parent [111] SensorSuite._read_altimeter

For the plane: perpendicular distance n·p and the direction −n toward the surface.

Receives	p: (3,) world (from [111])
Returns	(distance m, unit vector)

[111] SensorSuite._read_altimeter(p_ant, rot, terrain)

method · lander/sensors.py:71

Parent [109] SensorSuite.sample · L [110] PlaneTerrain.nearest

First-return radar altimeter (S3 p109): range to the nearest ground point from the antenna, discarded below 41 m (false echoes off the lander) or if the ground is not below the underside. Noise: (4.5 % + 1.5 m) / 3 × the noise scale (1σ).

Receives	p_ant (3), rot, terrain (from [109])
Returns	float (m), NaN = invalid
Calls	→ [110] **PlaneTerrain.nearest** — gets: p: (3,) world; returns: (distance m, unit vector)

[112] external `mujoco.mj_ray(model, data, pnt, vec, geomgroup, …)` — Intersects a ray with the geoms of the selected groups. Called by [113] PlaneTerrain.raycast — gets: point, direction, group mask; returns: distance or −1.

[113] PlaneTerrain.raycast(origin, direction)

method · lander/sim.py:181

Parent [114] SensorSuite._read_doppler · L [112] mujoco.mj_ray(model, data, pnt, vec, geomgroup, ...)

Distance along a ray to the ground, using MuJoCo's ray cast restricted to geom group 0 (terrain only).

Receives	origin (3,), direction (3,) (from [114])
Returns	float (m) or inf
Calls	→ [112] **mujoco.mj_ray(model, data, pnt, vec, geomgroup, ...)** — gets: point, direction, group mask; returns: distance or −1

[114] SensorSuite._read_doppler(p_ant, v_ant, rot, terrain)

method · lander/sensors.py:80

Parent [109] SensorSuite.sample · L [113] PlaneTerrain.raycast

Four-beam Doppler radar (S3 p111–112): each beam (21.5° from −Z, 45° clock) must hit the ground and see a closing speed of at least 1 m/s, so it has no lock while hovering. Noise: max(1.5 % × speed, 0.3 m/s) / 3 × the noise scale. With ≥ 3 locked beams the body-frame ground velocity is solved by least squares.

Receives	p_ant (3), v_ant (3), rot, terrain (from [109])
Returns	(beam speeds (4,), velocity (3,)) — NaN = no lock
Calls	→ [113] **PlaneTerrain.raycast** — gets: origin (3,), direction (3,); returns: float (m) or inf

[115] run_scenario(sim, scenario, on_step, settle_extra_s)

function · lander/scenarios.py:76

Parent [26] export_video.main · L [72] LanderSim.reset · R [140] VideoWriter.add, [143] hover_then_cut, [145] hover, [149] LanderSim.step, [171] TouchdownReport.done

The M1 episode loop. Resets the simulator with the scenario's seed, initial state and options, then repeatedly asks the policy for a Command and steps, until the scenario's duration or 1 s after the landing is decided. The optional `on_step` callback sees every StepResult (export_video renders a frame there).

Receives	sim, scenario, on_step(sim, StepResult) or None, settle_extra_s (from [26])
Returns	list[StepResult] — one per control step, including the reset
Calls	→ [72] **LanderSim.reset** — gets: seed (int or None = keep the last); initial_state: InitialState (None = 30 m hover); options: ResetOptions, dict or None; returns: StepResult (telemetry at t = 0, sensor readings, empty report, events []) → [140] **VideoWriter.add** — gets: res: StepResult; returns: None (count + 1) → [143] **hover_then_cut** — gets: sim; returns: Command (hover trim, engines_off = touched down) → [145] **hover** — gets: sim; returns: Command (hover trim) → [149] **LanderSim.step** — gets: command: Command (None = repeat the last); substep_callback(sim, k) or None; returns: StepResult → [171] ↺ **TouchdownReport.done** — gets: —; returns: bool: stop 1 s after this turns True

[116] external `Renderer.update_scene(data, camera)` — Builds the list of geoms to draw (MjvScene) for this state and camera. Called by [117] FrameRenderer.render — gets: data, camera; returns: None.

[117] FrameRenderer.render(res)

method · lander/video.py:59

Parent [140] VideoWriter.add · L [116] Renderer.update_scene(data, camera) · R [121] decorate, [126] Renderer.render(), [128] hud_lines, [139] overlay_hud

Updates MuJoCo's scene from the state and camera, adds plumes and arrows, renders to pixels, then draws the HUD.

Receives	res: StepResult or None (from [140])
Returns	ndarray (H, W, 3) uint8
Calls	→ [116] **Renderer.update_scene(data, camera)** — gets: data, camera; returns: None → [121] **decorate** — gets: scene (MjvScene), sim, plumes flag, arrows flag; returns: None (scene.ngeom grows) → [126] **Renderer.render()** — gets: —; returns: ndarray (H, W, 3) → [128] **hud_lines** — gets: sim, res: StepResult; returns: list of (label, value) strings → [139] **overlay_hud** — gets: img (H, W, 3), rows, font_size or None; returns: ndarray (H, W, 3)

[118] external `mujoco.mjv_initGeom(geom, type, size, pos, mat, rgba)` — Fills a scene geom's type, size, pose and colour. Called by [119] add_geom — gets: geom fields; returns: None.

[119] add_geom(scene, gtype, size, pos, mat, rgba)

function · lander/visuals.py:28

Parent [120] add_plumes · L [118] mujoco.mjv_initGeom(geom, type, size, pos, mat, rgba)

Appends one decorative geom to a scene if there is room and marks it as decor.

Receives	scene, geom type, size (3), position (3), rotation (3×3), rgba (4) (from [120])
Returns	bool (False when the scene is full)
Calls	→ [118] **mujoco.mjv_initGeom(geom, type, size, pos, mat, rgba)** — gets: geom fields; returns: None

[120] add_plumes(scene, sim)

function · lander/visuals.py:40

Parent [121] decorate · L [119] add_geom

For each engine, three nested translucent ellipsoids along the exhaust direction, length 3.5 m × (T / T_max)$^{0.7}$, width growing with thrust; engines below 1 % thrust draw nothing.

Receives	scene, sim — reads sim._last_thrust and the engine site poses (from [121])
Returns	None
Calls	→ [119] **add_geom** — gets: scene, geom type, size (3), position (3), rotation (3×3), rgba (4); returns: bool (False when the scene is full)

[121] decorate(scene, sim, plumes, arrows)

function · lander/visuals.py:67

Parent [117] FrameRenderer.render · L [120] add_plumes · R [122] LanderSim.telemetry, [124] add_arrow

Adds every overlay for the current state: engine plumes, a velocity arrow at the CG (green) and a wind arrow above it (blue).

Receives	scene (MjvScene), sim, plumes flag, arrows flag (from [117])
Returns	None (scene.ngeom grows)
Calls	→ [120] **add_plumes** — gets: scene, sim — reads sim._last_thrust and the engine site poses; returns: None → [122] ↺ **LanderSim.telemetry** — gets: —; returns: Telemetry (CG, velocity, wind) → [124] **add_arrow** — gets: scene, start (3), vec (3), rgba, width; returns: None

[122] ↺ **LanderSim.telemetry** reference: explained at [97]. Called by [121] decorate — gets: —; returns: Telemetry (CG, velocity, wind).

[123] _frame_with_z(z)

function · lander/visuals.py:19

Parent [124] add_arrow

Rotation matrix whose third column is the given direction (arrows point along their local +Z).

Receives	z (3,) (from [124])
Returns	ndarray (3, 3)

[124] add_arrow(scene, start, vec, rgba, width)

function · lander/visuals.py:59

Parent [121] decorate · L [123] _frame_with_z · R [125] add_geom

An arrow from `start` along `vec`, length = speed × 0.6 s (at most 8 m).

Receives	scene, start (3), vec (3), rgba, width (from [121])
Returns	None
Calls	→ [123] **_frame_with_z** — gets: z (3,); returns: ndarray (3, 3) → [125] ↺ **add_geom** — gets: arrow geom; returns: bool

[125] ↺ **add_geom** reference: explained at [119]. Called by [124] add_arrow — gets: arrow geom; returns: bool.

[126] external `Renderer.render()` — Draws the scene offscreen and reads the pixels back. Called by [117] FrameRenderer.render — gets: —; returns: ndarray (H, W, 3).

[127] SensorReadings.altitude_valid

property · lander/sensors.py:37

Parent [128] hud_lines

True when the altimeter reading is a number (not NaN).

Receives	(self) (from [128])
Returns	bool

[128] hud_lines(sim, res)

function · lander/hud.py:28

Parent [117] FrameRenderer.render · L [127] SensorReadings.altitude_valid · R [129] wind_heading_deg, [130] Telemetry.vz, [131] Telemetry.vh, [132] Telemetry.spin_dps, [134] status_text

The HUD rows (brief §8): time, radar altitude ('--' when the altimeter has no valid reading, e.g. below 41 m), true footpad altitude, vertical and horizontal speed, tilt, spin, propellant, each engine's thrust as % of maximum, engines ON / CUT, wind ('calm' or speed and heading) and a status line.

Receives	sim, res: StepResult (from [117])
Returns	list of (label, value) strings
Calls	→ [127] **SensorReadings.altitude_valid** — gets: (self); returns: bool → [129] **wind_heading_deg** — gets: wind (3,); returns: float (deg) → [130] **Telemetry.vz** — gets: (self); returns: float (m/s) → [131] **Telemetry.vh** — gets: (self); returns: float (m/s) → [132] **Telemetry.spin_dps** — gets: (self); returns: float (deg/s) → [134] **status_text** — gets: res: StepResult; returns: str

[129] wind_heading_deg(wind)

function · lander/visuals.py:100

Parent [128] hud_lines

Direction the wind blows toward, 0–360° from +X toward +Y.

Receives	wind (3,) (from [128])
Returns	float (deg)

[130] Telemetry.vz

property · lander/sim.py:141

Parent [128] hud_lines

Vertical CG speed (+ up).

Receives	(self) (from [128])
Returns	float (m/s)

[131] Telemetry.vh

property · lander/sim.py:145

Parent [128] hud_lines

Horizontal CG speed.

Receives	(self) (from [128])
Returns	float (m/s)

[132] Telemetry.spin_dps

property · lander/sim.py:149

Parent [128] hud_lines

Rotation rate about the lander's own vertical (thrust) axis. Nothing controls it (no roll jets, D4).

Receives	(self) (from [128])
Returns	float (deg/s)

[133] ↺ **TouchdownReport.done** reference: explained at [163]. Called by [134] status_text — gets: —; returns: bool.

[134] status_text(res)

function · lander/hud.py:10

Parent [128] hud_lines · L [133] TouchdownReport.done · R [136] TouchdownReport.success, [137] TouchdownReport.failures

One status line: 'LANDED - all section 7 checks pass', 'DOWN - engines still firing (must be cut)', 'FAILED: leg broken at …', 'FAILED: <criteria>', 'TOUCHDOWN <speeds>', 'ENGINES OFF' or 'POWERED DESCENT'.

Receives	res: StepResult (from [128])
Returns	str
Calls	→ [133] ↺ **TouchdownReport.done** — gets: —; returns: bool → [136] **TouchdownReport.success** — gets: (self); returns: bool → [137] **TouchdownReport.failures** — gets: —; returns: list[str]

[135] ↺ **TouchdownReport.done** reference: explained at [163]. Called by [136] TouchdownReport.success — gets: —; returns: bool.

[136] TouchdownReport.success

property · lander/touchdown.py:70

Parent [134] status_text · L [135] TouchdownReport.done

Decided, and every judged criterion passed (criteria still None, the O3 limits, are skipped).

Receives	(self) (from [134])
Returns	bool
Calls	→ [135] ↺ **TouchdownReport.done** — gets: —; returns: bool

[137] TouchdownReport.failures()

method · lander/touchdown.py:73

Parent [134] status_text

Names of the criteria that are False.

Receives	— (from [134])
Returns	list[str]

[138] _font(size)

function · lander/video.py:20

Parent [139] overlay_hud

Pillow's built-in scalable font at a size (bitmap font on very old Pillow).

Receives	size (px) (from [139])
Returns	ImageFont

[139] overlay_hud(img, rows, font_size)

function · lander/video.py:27

Parent [117] FrameRenderer.render · L [138] _font

Draws the rows in a translucent top-left panel (labels amber, values white) and the last row, the status, as a banner at the bottom centre. The font size scales with the image height.

Receives	img (H, W, 3), rows, font_size or None (from [117])
Returns	ndarray (H, W, 3)
Calls	→ [138] **_font** — gets: size (px); returns: ImageFont

[140] VideoWriter.add(res)

method · lander/video.py:83

Parent [115] run_scenario · L [117] FrameRenderer.render · R [141] writer.append_data(frame)

Renders the current state and appends the frame to the MP4. `run_scenario` reaches it through the `on_step` callback that export_video.main passes in.

Receives	res: StepResult (from [115])
Returns	None (count + 1)
Note	called through the on_step lambda
Calls	→ [117] **FrameRenderer.render** — gets: res: StepResult or None; returns: ndarray (H, W, 3) uint8 → [141] **writer.append_data(frame)** — gets: frame (H, W, 3); returns: None

[141] external `writer.append_data(frame)` — Encodes one frame into the MP4. Called by [140] VideoWriter.add — gets: frame (H, W, 3); returns: None.

[142] ↺ **LanderSim.hover_trim** reference: explained at [83]. Called by [143] hover_then_cut — gets: — (current propellant); returns: 3 valve settings.

[143] hover_then_cut(sim)

function · lander/scenarios.py:25

Parent [115] run_scenario · L [142] LanderSim.hover_trim

The policy of drop, hard_drop, slope45 and freefall: thrust = weight with zero pitch and yaw torque until the first footpad contact, then engines off for good. It uses the latched touchdown flag because the rigid legs rebound slightly (~13 % of the impact speed). Nothing corrects attitude or drift, so the scenarios show the raw physics.

Receives	sim (from [115])
Returns	Command (hover trim, engines_off = touched down)
Note	called as scenario.policy(sim)
Calls	→ [142] ↺ **LanderSim.hover_trim** — gets: — (current propellant); returns: 3 valve settings

[144] ↺ **LanderSim.hover_trim** reference: explained at [83]. Called by [145] hover — gets: — (current propellant); returns: 3 valve settings.

[145] hover(sim)

function · lander/scenarios.py:20

Parent [115] run_scenario · L [144] LanderSim.hover_trim

The hover scenario's policy: thrust = weight, engines never cut. With no attitude control the thrust misalignment slowly tilts and spins the lander.

Receives	sim (from [115])
Returns	Command (hover trim)
Note	called as scenario.policy(sim) for the hover scenario
Calls	→ [144] ↺ **LanderSim.hover_trim** — gets: — (current propellant); returns: 3 valve settings

[146] ↺ **EngineBank.valve_to_thrust** reference: explained at [86]. Called by [147] EngineBank.command — gets: valve settings; returns: target thrusts (N).

[147] EngineBank.command(throttle, engines_off)

method · lander/vehicle.py:85

Parent [148] Vehicle.command · L [146] EngineBank.valve_to_thrust

New thrust targets from the valve settings; `engines_off` shuts the engines down for good.

Receives	throttle (3), engines_off (from [148])
Returns	None (cmd updated; lit may become False)
Calls	→ [146] ↺ **EngineBank.valve_to_thrust** — gets: valve settings; returns: target thrusts (N)

[148] Vehicle.command(cmd)

method · lander/vehicle.py:131

Parent [149] LanderSim.step · L [147] EngineBank.command

Hands the command to the engines at the start of the control period.

Receives	cmd: Command (throttle (3), engines_off) (from [149])
Returns	None
Calls	→ [147] **EngineBank.command** — gets: throttle (3), engines_off; returns: None (cmd updated; lit may become False)

[149] LanderSim.step(command, substep_callback)

method · lander/sim.py:309

Parent [115] run_scenario · L [148] Vehicle.command · R [151] LanderSim._physics_step, [169] mujoco.mj_forward(model, data), [170] LanderSim._result

Advances one control period (40 ms): latches the command (zero-order hold), runs 20 physics steps, recomputes MuJoCo's derived quantities for the final state and returns the outputs.

Receives	command: Command (None = repeat the last); substep_callback(sim, k) or None (from [115])
Returns	StepResult
Calls	→ [148] **Vehicle.command** — gets: cmd: Command (throttle (3), engines_off); returns: None → [151] **LanderSim._physics_step** — gets: events: list (appended in place); reads data.qpos and data.qvel; returns: None → [169] **mujoco.mj_forward(model, data)** — gets: model, data; returns: None → [170] ↺ **LanderSim._result** — gets: events of this control step; returns: StepResult

[150] ↺ **quat_to_mat** reference: explained at [61]. Called by [151] LanderSim._physics_step — gets: data.qpos[3:7]; returns: rotation R.

[151] LanderSim._physics_step(events)

method · lander/sim.py:323

Parent [149] LanderSim.step · L [150] quat_to_mat · R [152] cross3, [154] Vehicle.step, [155] drag_force_body, [156] LanderSim._apply_mass, [157] mujoco.mj_step(model, data), [158] LanderSim._contacts, [160] LanderSim._flag_events, [161] LanderSim.engines_off, [162] TouchdownMonitor.note_engines, [163] TouchdownReport.done, [164] stability_distance, [165] tilt_of, [167] TouchdownMonitor.update

One 2 ms physics step, in this order: read the state (rotation, CG, CG velocity) → engines and propellant → drag in the steady wind → write the applied force and torque into MuJoCo → update the mass → mj_step → contacts and events → note the engine state → touchdown monitor (only near the ground, after touchdown, or once the tank is empty). Figure 2 shows the data flow.

Receives	events: list (appended in place); reads data.qpos and data.qvel (from [149])
Returns	None
State	writes data.xfrc_applied (force + torque on the lander body, world frame) and the model mass; caches _last_thrust
Calls	→ [150] ↺ **quat_to_mat** — gets: data.qpos[3:7]; returns: rotation R → [152] ↺ **cross3** — gets: ω_world × R·cg; returns: CG velocity offset → [154] **Vehicle.step** — gets: dt; returns: VehicleStep (thrust (3), force_b, torque_b, mdot, empty) → [155] **drag_force_body** — gets: v_rel_body (3), rho, cda (3); returns: ndarray (3,) N, body frame → [156] ↺ **LanderSim._apply_mass** — gets: propellant left; returns: None — mass = dry + propellant → [157] **mujoco.mj_step(model, data)** — gets: model, data; returns: None (new qpos, qvel, contacts, sensordata) → [158] **LanderSim._contacts** — gets: — reads data.contact; returns: None → [160] **LanderSim._flag_events** — gets: events list; returns: None (events appended) → [161] ↺ **LanderSim.engines_off** — gets: —; returns: bool for note_engines → [162] ↺ **TouchdownMonitor.note_engines** — gets: engines_off; returns: None → [163] **TouchdownReport.done** — gets: (self); returns: bool → [164] **stability_distance** — gets: cg (3), pads (3, 3) world; returns: float (m) → [165] ↺ **tilt_of** — gets: rotation R; returns: tilt (deg) → [167] **TouchdownMonitor.update** — gets: t, pad_contact (3), body_contact, v_cg (3), ke (J), stability (m), tilt_deg, empty, xy; returns: None (report updated)

[152] ↺ **cross3** reference: explained at [98]. Called by [151] LanderSim._physics_step — gets: ω_world × R·cg; returns: CG velocity offset.

[153] EngineBank.step(dt, cut)

method · lander/vehicle.py:90

Parent [154] Vehicle.step

Per engine: first-order lag toward the target with τ = 23 ms (the same up and down), clamp to [T_min, T_max] while lit or [0, T_max] once shut down, then the fault scale. A cut latches the engines off for good.

Receives	dt, cut (from [154])
Returns	actual thrust (3) N

[154] Vehicle.step(dt)

method · lander/vehicle.py:134

Parent [151] LanderSim._physics_step · L [153] EngineBank.step

Advances the engines (cut once the tank is empty), integrates propellant use from the actual thrust (D6), and sums the thrust force and its torque about the CG in body axes.

Receives	dt (from [151])
Returns	VehicleStep (thrust (3), force_b, torque_b, mdot, empty)
State	decreases propellant; sets empty
Calls	→ [153] **EngineBank.step** — gets: dt, cut; returns: actual thrust (3) N

[155] drag_force_body(v_rel_body, rho, cda)

function · lander/aero.py:18

Parent [151] LanderSim._physics_step

Quadratic drag per body axis: $F_k = -\tfrac{1}{2}\,\rho\,|v|\,(C_dA)_k\,v_k$, with v the CG velocity relative to the steady wind in body axes and ρ = 0.016 kg/m³ at every height. It acts at the CG (no torque).

Receives	v_rel_body (3), rho, cda (3) (from [151])
Returns	ndarray (3,) N, body frame

[156] ↺ **LanderSim._apply_mass** reference: explained at [79]. Called by [151] LanderSim._physics_step — gets: propellant left; returns: None — mass = dry + propellant.

[157] external `mujoco.mj_step(model, data)` — MuJoCo advances 2 ms: forward dynamics with the applied force and torque, gravity and soft contacts (implicitfast), then integrates qpos and qvel and fills sensordata. Called by [151] LanderSim._physics_step — gets: model, data; returns: None (new qpos, qvel, contacts, sensordata).

[158] LanderSim._contacts()

method · lander/sim.py:366

Parent [151] LanderSim._physics_step

Scans MuJoCo's contact list for pairs that involve the terrain: footpad spheres → pad_contact (3 flags); hull, radar housing, engines or tanks → body_contact.

Receives	— reads data.contact (from [151])
Returns	None
State	sets _pad_contact, _body_contact

[159] ↺ **LanderSim.engines_off** reference: explained at [92]. Called by [160] LanderSim._flag_events — gets: —; returns: bool.

[160] LanderSim._flag_events(events)

method · lander/sim.py:378

Parent [151] LanderSim._physics_step · L [159] LanderSim.engines_off

Adds each event once per episode: engines_cut, propellant_exhausted, body_strike.

Receives	events list (from [151])
Returns	None (events appended)
Calls	→ [159] ↺ **LanderSim.engines_off** — gets: —; returns: bool

[161] ↺ **LanderSim.engines_off** reference: explained at [92]. Called by [151] LanderSim._physics_step — gets: —; returns: bool for note_engines.

[162] ↺ **TouchdownMonitor.note_engines** reference: explained at [94]. Called by [151] LanderSim._physics_step — gets: engines_off; returns: None.

[163] TouchdownReport.done

property · lander/touchdown.py:65

Parent [151] LanderSim._physics_step

The landing is decided: settled, tipped over, a broken leg, or the body hit the ground after touchdown.

Receives	(self) (from [151])
Returns	bool

[164] stability_distance(cg, pads)

function · lander/touchdown.py:25

Parent [151] LanderSim._physics_step

TN D-7959 stability measure: the smallest signed horizontal distance from the CG to the vertical planes through each pair of footpads. Positive inside the footpad triangle, negative = tipping over.

Receives	cg (3), pads (3, 3) world (from [151])
Returns	float (m)

[165] ↺ **tilt_of** reference: explained at [103]. Called by [151] LanderSim._physics_step — gets: rotation R; returns: tilt (deg).

[166] ↺ **TouchdownMonitor._judge** reference: explained at [93]. Called by [167] TouchdownMonitor.update — gets: —; returns: None.

[167] TouchdownMonitor.update(t, pad_contact, body_contact, v_cg, ke, stability, tilt_deg, empty, xy)

method · lander/touchdown.py:98

Parent [151] LanderSim._physics_step · L [166] TouchdownMonitor._judge · R [168] TouchdownReport.done

Before contact it only watches for an empty tank or a body strike. At first contact it records the speeds and tilt, decides whether a leg broke (faster than 3.34 m/s down or 1.22 m/s sideways, S5) and keeps the impact kinetic energy (floor 100 J). After that it tracks the minimum stability distance, flags a tip-over (stability < 0) and declares the lander settled when it rests on all three feet with kinetic energy below 0.5 % of the impact energy for 0.5 s. Then it re-judges the criteria.

Receives	t, pad_contact (3), body_contact, v_cg (3), ke (J), stability (m), tilt_deg, empty, xy (from [151])
Returns	None (report updated)
Calls	→ [166] ↺ **TouchdownMonitor._judge** — gets: —; returns: None → [168] ↺ **TouchdownReport.done** — gets: —; returns: bool: is the landing decided?

[168] ↺ **TouchdownReport.done** reference: explained at [163]. Called by [167] TouchdownMonitor.update — gets: —; returns: bool: is the landing decided?.

[169] external `mujoco.mj_forward(model, data)` — Brings positions and sensor data up to date for the final state of the control period. Called by [149] LanderSim.step — gets: model, data; returns: None.

[170] ↺ **LanderSim._result** reference: explained at [105]. Called by [149] LanderSim.step — gets: events of this control step; returns: StepResult.

[171] ↺ **TouchdownReport.done** reference: explained at [163]. Called by [115] run_scenario — gets: —; returns: bool: stop 1 s after this turns True.

[172] FrameRenderer.close()

method · lander/video.py:67

Parent [173] VideoWriter.close

Releases the OpenGL context.

Receives	— (from [173])
Returns	None

[173] VideoWriter.close()

method · lander/video.py:87

Parent [26] export_video.main · L [172] FrameRenderer.close

Finishes the MP4 file and releases the renderer.

Receives	— (from [26])
Returns	None
Calls	→ [172] **FrameRenderer.close** — gets: —; returns: None

[174] ↺ **TouchdownReport.success** reference: explained at [136]. Called by [175] summarize — gets: —; returns: bool.

[175] summarize(report, telemetry)

function · scripts/common.py:40

Parent [26] export_video.main · L [174] TouchdownReport.success · R [176] TouchdownReport.done, [177] TouchdownReport.failures

Readable multi-line outcome: time, propellant left and engine state; touchdown speeds and tilt (and 'LEG BROKEN'); minimum stability distance, settled, tipped or body strike; each criterion (n/a for the M2 ones); RESULT: SUCCESS, FAILED: <criteria> or undecided.

Receives	report: TouchdownReport, telemetry: Telemetry (from [26])
Returns	str
Calls	→ [174] ↺ **TouchdownReport.success** — gets: —; returns: bool → [176] ↺ **TouchdownReport.done** — gets: —; returns: bool → [177] ↺ **TouchdownReport.failures** — gets: —; returns: list[str]

[176] ↺ **TouchdownReport.done** reference: explained at [163]. Called by [175] summarize — gets: —; returns: bool.

[177] ↺ **TouchdownReport.failures** reference: explained at [137]. Called by [175] summarize — gets: —; returns: list[str].

[178] ↺ **add_wind_arg** reference: explained at [25]. Called by [179] watch.main — gets: the script's parser; returns: None.

[179] watch.main(argv)

function · scripts/watch.py:21

Parent [2] main · L [178] add_wind_arg · R [180] add_config_args, [181] config_from_args, [182] LanderSim.__init__, [183] make_scenario, [184] wind_options, [187] watch, [198] LanderSim.telemetry, [199] summarize

Watches a scenario live in MuJoCo's viewer window; the scenario's policy flies it (the RL agent will plug in here at M4/M5). Prints the landing summary when the window closes.

Receives	argv → --scenario, --seed, --speed, --max-seconds, --no-hud, --wind, --set, --env-file (from [2])
Returns	None — a window, then a printed summary
Calls	→ [178] ↺ **add_wind_arg** — gets: the script's parser; returns: None → [180] ↺ **add_config_args** — gets: the script's parser; returns: None → [181] ↺ **config_from_args** — gets: parsed args; returns: Config → [182] ↺ **LanderSim.__init__** — gets: cfg, seed; returns: LanderSim → [183] ↺ **make_scenario** — gets: scenario name, sim, seed; returns: Scenario → [184] ↺ **wind_options** — gets: parsed args; returns: reset options added to the scenario's → [187] **watch** — gets: sim, policy(sim) → Command, reset_fn() → StepResult, speed, max_seconds, hud, hold_after_done_s; returns: None — returns when the window closes or max_seconds have passed → [198] ↺ **LanderSim.telemetry** — gets: —; returns: Telemetry for the summary → [199] ↺ **summarize** — gets: final report, telemetry; returns: str

[180] ↺ **add_config_args** reference: explained at [3]. Called by [179] watch.main — gets: the script's parser; returns: None.

[181] ↺ **config_from_args** reference: explained at [6]. Called by [179] watch.main — gets: parsed args; returns: Config.

[182] ↺ **LanderSim.__init__** reference: explained at [30]. Called by [179] watch.main — gets: cfg, seed; returns: LanderSim.

[183] ↺ **make_scenario** reference: explained at [63]. Called by [179] watch.main — gets: scenario name, sim, seed; returns: Scenario.

[184] ↺ **wind_options** reference: explained at [64]. Called by [179] watch.main — gets: parsed args; returns: reset options added to the scenario's.

[185] ↺ **LanderSim.reset** reference: explained at [72]. Called by [186] reset_fn (in watch.main) — gets: scenario seed, initial_state, options; returns: StepResult.

[186] reset_fn (in watch.main)()

nested function · scripts/watch.py:38

Parent [187] watch · L [185] LanderSim.reset

Defined in watch.main: resets the simulator with the scenario's seed, initial state and options, and remembers the result.

Receives	— (from [187])
Returns	StepResult
Calls	→ [185] ↺ **LanderSim.reset** — gets: scenario seed, initial_state, options; returns: StepResult

[187] watch(sim, policy, reset_fn, speed, max_seconds, hud, hold_after_done_s)

function · lander/viewer.py:22

Parent [179] watch.main · L [186] reset_fn (in watch.main) · R [188] mujoco.viewer.launch_passive(model, data, key_callback, ...), [189] on_key (in watch), [190] tracking_camera, [192] policy (in watch.main), [193] LanderSim.step, [194] decorate, [196] hud_columns, [197] handle.set_texts(...) / handle.sync()

The live viewer loop. Resets via reset_fn, opens MuJoCo's passive viewer with a key callback (Space pause, Enter restart), points the camera at the lander, then each iteration: restart if asked; step the policy unless paused or holding after the landing is decided (2 s); redraw the overlays into the viewer's user scene; show the HUD with set_texts; sync; sleep so that simulated time runs at speed × real time.

Receives	sim, policy(sim) → Command, reset_fn() → StepResult, speed, max_seconds, hud, hold_after_done_s (from [179])
Returns	None — returns when the window closes or max_seconds have passed
Calls	→ [186] **reset_fn (in watch.main)** — gets: —; returns: StepResult → [188] **mujoco.viewer.launch_passive(model, data, key_callback, ...)** — gets: model, data, key_callback; returns: viewer handle → [189] **on_key (in watch)** — gets: key: GLFW key code; returns: None → [190] ↺ **tracking_camera** — gets: sim; returns: camera settings copied into the viewer → [192] **policy (in watch.main)** — gets: sim; returns: Command → [193] ↺ **LanderSim.step** — gets: the policy's Command; returns: StepResult → [194] ↺ **decorate** — gets: viewer.user_scn, sim; returns: None → [196] **hud_columns** — gets: sim, res; returns: (str, str) → [197] **handle.set_texts(...) / handle.sync()** — gets: texts; returns: None

[188] external `mujoco.viewer.launch_passive(model, data, key_callback, ...)` — Opens the viewer window on its own thread and returns a handle (lock, user_scn, cam, set_texts, sync, is_running). Called by [187] watch — gets: model, data, key_callback; returns: viewer handle.

[189] on_key (in watch)(key)

nested function · lander/viewer.py:32

Parent [187] watch

Runs on the viewer's thread: Space toggles pause, Enter asks for a restart. It only sets flags; the main loop acts on them.

Receives	key: GLFW key code (from [187])
Returns	None

[190] ↺ **tracking_camera** reference: explained at [67]. Called by [187] watch — gets: sim; returns: camera settings copied into the viewer.

[191] ↺ **hover_then_cut** reference: explained at [143]. Called by [192] policy (in watch.main) — gets: sim; returns: Command.

[192] policy (in watch.main)(s)

nested function · scripts/watch.py:42

Parent [187] watch · L [191] hover_then_cut

Defined in watch.main: forwards to the scenario's policy.

Receives	sim (from [187])
Returns	Command
Calls	→ [191] ↺ **hover_then_cut** — gets: sim; returns: Command

[193] ↺ **LanderSim.step** reference: explained at [149]. Called by [187] watch — gets: the policy's Command; returns: StepResult.

[194] ↺ **decorate** reference: explained at [121]. Called by [187] watch — gets: viewer.user_scn, sim; returns: None.

[195] ↺ **hud_lines** reference: explained at [128]. Called by [196] hud_columns — gets: sim, res; returns: rows.

[196] hud_columns(sim, res)

function · lander/hud.py:52

Parent [187] watch · L [195] hud_lines

The HUD rows as two newline-joined columns (labels, values) for MuJoCo's text overlay.

Receives	sim, res (from [187])
Returns	(str, str)
Calls	→ [195] ↺ **hud_lines** — gets: sim, res; returns: rows

[197] external `handle.set_texts(…) / handle.sync()` — Overlays the HUD text and copies the latest state to the window. Called by [187] watch — gets: texts; returns: None.

[198] ↺ **LanderSim.telemetry** reference: explained at [97]. Called by [179] watch.main — gets: —; returns: Telemetry for the summary.

[199] ↺ **summarize** reference: explained at [175]. Called by [179] watch.main — gets: final report, telemetry; returns: str.

[200] ↺ **LanderSim.__init__** reference: explained at [30]. Called by [201] _worker — gets: seed (cfg loaded inside); returns: LanderSim.

[201] _worker(sim_seconds, seed)

function · scripts/benchmark.py:17

Parent [206] run · L [200] LanderSim.__init__ · R [202] make_scenario, [203] LanderSim.reset, [204] hover, [205] LanderSim.step

In its own process: builds a LanderSim, flies the hover scenario again and again (a new episode every 20 simulated seconds) until sim_seconds have been simulated, and times it.

Receives	sim_seconds, seed (from [206])
Returns	(control steps, wall seconds)
Calls	→ [200] ↺ **LanderSim.__init__** — gets: seed (cfg loaded inside); returns: LanderSim → [202] ↺ **make_scenario** — gets: 'hover', sim, seed; returns: Scenario → [203] ↺ **LanderSim.reset** — gets: seed + steps, hover state; returns: StepResult → [204] ↺ **hover** — gets: sim; returns: Command → [205] ↺ **LanderSim.step** — gets: Command; returns: StepResult

[202] ↺ **make_scenario** reference: explained at [63]. Called by [201] _worker — gets: 'hover', sim, seed; returns: Scenario.

[203] ↺ **LanderSim.reset** reference: explained at [72]. Called by [201] _worker — gets: seed + steps, hover state; returns: StepResult.

[204] ↺ **hover** reference: explained at [145]. Called by [201] _worker — gets: sim; returns: Command.

[205] ↺ **LanderSim.step** reference: explained at [149]. Called by [201] _worker — gets: Command; returns: StepResult.

[206] run(procs, sim_seconds)

function · scripts/benchmark.py:34

Parent [207] benchmark.main · L [201] _worker

Starts a pool of N spawned processes running _worker and aggregates: total control steps ÷ the slowest worker's wall time.

Receives	procs, sim_seconds (from [207])
Returns	(control steps per second, real-time factor)
Calls	→ [201] **_worker** — gets: sim_seconds, seed; returns: (control steps, wall seconds)

[207] benchmark.main(argv)

function · scripts/benchmark.py:44

Parent [2] main · L [206] run

Measures simulator speed with 1, 2, 4, 6, 8 worker processes and prints control steps per second, the real-time factor and the time 10 M environment steps would take.

Receives	argv → --procs, --sim-seconds (from [2])
Returns	None — printed table
Calls	→ [206] **run** — gets: procs, sim_seconds; returns: (control steps per second, real-time factor)

[208] ↺ **_override** reference: explained at [11]. Called by [209] Config.with_overrides — gets: old Param, value, origin; returns: override Param.

[209] Config.with_overrides(overrides, origin)

method · lander/config.py:205

Parent [211] tests/conftest.py · L [208] _override · R [210] build_config

A new Config with some values replaced: each becomes an `override` Param with the given origin, then everything is rebuilt and re-validated.

Receives	overrides: dict key → value, origin (from [211])
Returns	Config
Calls	→ [208] ↺ **_override** — gets: old Param, value, origin; returns: override Param → [210] ↺ **build_config** — gets: updated params; returns: Config

[210] ↺ **build_config** reference: explained at [13]. Called by [209] Config.with_overrides — gets: updated params; returns: Config.

[211] tests/conftest.py

test file

Parent [212] tests (python -m pytest) · L [209] Config.with_overrides

Fixtures shared by all tests: `cfg` = load_config(environ={}) so local LANDER_* overrides cannot change results; `make_sim(overrides, seed)` = LanderSim(cfg.with_overrides(...)); `CLEAN` = no air, no misalignment and perfect radar readings, for comparisons with closed-form physics.

Receives	— (from [212])
Returns	fixtures
Calls	→ [209] **Config.with_overrides** — gets: overrides: dict key → value, origin; returns: Config

[212] tests (python -m pytest)

test suite

Parent [2] main · L [211] tests/conftest.py · R [214] tests/test_config.py, [217] tests/test_model.py, [218] tests/test_physics.py, [219] tests/test_legs.py, [221] tests/test_aero_sensors.py, [222] tests/test_touchdown.py, [223] tests/test_video_docs.py

The test suite: 37 headless tests in 7 files (≈ 25 s). Tests call the library directly rather than the scripts. The children are the test files; library functions that only the tests use are explained under them.

Receives	pytest command line (from [2])
Returns	pass / fail report
Calls	→ [211] **tests/conftest.py** — gets: —; returns: fixtures → [214] **tests/test_config.py** — gets: —; returns: 9 tests → [217] **tests/test_model.py** — gets: —; returns: 4 tests → [218] **tests/test_physics.py** — gets: —; returns: 7 tests → [219] **tests/test_legs.py** — gets: —; returns: 2 tests → [221] **tests/test_aero_sensors.py** — gets: —; returns: 6 tests → [222] **tests/test_touchdown.py** — gets: —; returns: 6 tests → [223] **tests/test_video_docs.py** — gets: —; returns: 3 tests

[213] Config.param(key)

method · lander/config.py:182

Parent [214] tests/test_config.py

The full record of one key (`ConfigError` if unknown).

Receives	key (from [214])
Returns	Param

[214] tests/test_config.py

test file

Parent [212] tests (python -m pytest) · L [213] Config.param · R [215] Config.overridden, [216] Config.to_log_dict

The fact file: strict parsing, metadata rules, parked facts ignored, overrides, and the derived numbers.

Tests: `test_every_fact_has_status_and_source`, `test_parked_facts_are_comments_only`, `test_missing_metadata_is_rejected`, `test_bad_status_is_rejected`, `test_published_value_needs_a_source`, `test_unknown_and_missing_keys_are_rejected`, `test_env_var_override_is_recorded`, `test_cli_override_and_errors`, `test_derived_numbers_match_the_brief`

Receives	— (from [212])
Returns	9 tests
Calls	→ [213] **Config.param** — gets: key; returns: Param → [215] **Config.overridden** — gets: —; returns: dict key → Param → [216] **Config.to_log_dict** — gets: —; returns: dict

[215] Config.overridden()

method · lander/config.py:192

Parent [214] tests/test_config.py

The params whose value came from an override.

Receives	— (from [214])
Returns	dict key → Param

[216] Config.to_log_dict()

method · lander/config.py:196

Parent [214] tests/test_config.py

JSON-ready dump of every param (value, unit, status, source, note, origin), for M2's episode logs.

Receives	— (from [214])
Returns	dict

[217] tests/test_model.py

test file

Parent [212] tests (python -m pytest)

The generated MuJoCo model is one free body; mass = dry + propellant at every step while the CG and inertia stay fixed; the footprint matches the drawings; editing .env changes the compiled model.

Tests: `test_model_dimensions`, `test_mass_is_dry_plus_propellant_with_fixed_cg_and_inertia`, `test_footprint_matches_drawings`, `test_model_is_built_from_env`

Receives	— (from [212])
Returns	4 tests

[218] tests/test_physics.py

test file

Parent [212] tests (python -m pytest)

Brief §11: free fall vs closed form, hover holds altitude and stays level, misalignment tilts an uncontrolled lander, mass flow = $\int T / (Isp \cdot g_0)\, dt$, the engines-off command is permanent, bit-identical determinism, rest stability on flat ground (the dt check).

Tests: `test_free_fall_matches_closed_form`, `test_hover_holds_altitude_and_stays_level`, `test_misalignment_tilts_an_uncontrolled_lander`, `test_mass_flow_equals_thrust_integral`, `test_engines_off_command_is_permanent`, `test_determinism_same_seed_bit_identical`, `test_rest_on_flat_ground_is_stable`

Receives	— (from [212])
Returns	7 tests

[219] tests/test_legs.py

test file

Parent [212] tests (python -m pytest)

Rigid legs: a leg breaks above 3.34 m/s down or 1.22 m/s sideways and not below; slope drops (45° tips over, 10° never; NASA's 0.85 printed for comparison).

Tests: `test_leg_breaks_above_the_design_speeds`, `test_steep_slopes_tip_over_gentle_slopes_do_not`

Receives	— (from [212])
Returns	2 tests

[220] SensorReadings.doppler_valid

property · lander/sensors.py:41

Parent [221] tests/test_aero_sensors.py

Mask of Doppler beams with a lock (finite readings).

Receives	(self) (from [221])
Returns	bool array (4,)

[221] tests/test_aero_sensors.py

test file

Parent [212] tests (python -m pytest) · L [220] SensorReadings.doppler_valid

Drag in calm descent and in a dust storm, storm drift after 30 s, ideal radar altimeter (41 m cut-off), ideal Doppler (no lock in hover), ideal IMU, radar noise at the published levels.

Tests: `test_drag_calm_versus_storm`, `test_storm_drift_matches_the_review`, `test_ideal_radar_altimeter_and_cutoff`, `test_ideal_doppler_radar`, `test_ideal_imu`, `test_radar_noise_uses_published_levels`

Receives	— (from [212])
Returns	6 tests
Calls	→ [220] **SensorReadings.doppler_valid** — gets: (self); returns: bool array (4,)

[222] tests/test_touchdown.py

test file

Parent [212] tests (python -m pytest)

Stability-distance geometry; the nominal drop succeeds with events in order; the hard drop breaks a leg; free fall crashes; a landing only counts once the engines are cut; running out of propellant in the air is flagged.

Tests: `test_stability_distance_geometry`, `test_nominal_drop_succeeds`, `test_hard_drop_breaks_a_leg`, `test_free_fall_crashes`, `test_engines_must_be_cut_before_the_landing_counts`, `test_out_of_propellant_is_flagged`

Receives	— (from [212])
Returns	6 tests

[223] tests/test_video_docs.py

test file

Parent [212] tests (python -m pytest)

Offscreen frame and MP4 work; README names every module, class and function; the vehicle layer never imports MuJoCo.

Tests: `test_offscreen_render_and_mp4`, `test_readme_explains_every_file_and_function`, `test_vehicle_layer_does_not_import_mujoco`

Receives	— (from [212])
Returns	3 tests

[224] notebooks/m1_physics_tour.ipynb

notebook

Parent [2] main

Explores and plots the physics checks; all logic lives in the package. It uses `load_config` and `derived`, `EngineBank` (valve step and the engines-off command), `LanderSim` (misalignment drift, storm drift, free fall, mass flow), `drag_areas` and `drag_force_body` (the storm push), `run_scenario` (the drop) and `export_video.main` (embedded video).

Receives	a Jupyter kernel running the .venv (from [2])
Returns	6 figures and an embedded MP4

Appendix A Functions on no call path from main

These are kept for the next milestones or as a small public API; nothing in the scripts calls them yet (tests may).

function	where	what it is for
`Config.by_status`	lander/config.py:188	All params with the given status, e.g. 'assumed' (the ones to verify first).

Appendix B Where each file's functions are explained

file	nodes (in-order number and name)
`lander/aero.py`	[54] drag_areas, [89] wind_vector, [155] drag_force_body
`lander/config.py`	[5] parse_set_args, [7] parse_value, [8] parse_env_file, [9] load_config, [11] _override, [12] _group_of, [13] build_config, [14] _coerce, [15] SimCfg.control_dt_s, [16] SimCfg.substeps, [17] validate, [22] derived, [209] Config.with_overrides, [213] Config.param, [215] Config.overridden, [216] Config.to_log_dict
`lander/geometry.py`	[31] unit_azimuth, [32] build_geometry, [37] hexagon_vertices, [98] cross3
`lander/hud.py`	[128] hud_lines, [134] status_text, [196] hud_columns
`lander/mjcf.py`	[34] build_mjcf, [38] _v, [39] _zquat, [41] build_model, [43] build_model.gid, [44] build_model.sensor_adr, [56] terrain_normal, [75] set_terrain, [76] quat_z_to
`lander/scenarios.py`	[63] make_scenario, [115] run_scenario, [143] hover_then_cut, [145] hover
`lander/sensors.py`	[50] SensorSuite.reset, [51] SensorSuite.__init__, [109] SensorSuite.sample, [111] SensorSuite._read_altimeter, [114] SensorSuite._read_doppler, [127] SensorReadings.altitude_valid, [220] SensorReadings.doppler_valid
`lander/sim.py`	[30] LanderSim.__init__, [53] PlaneTerrain.__init__, [57] LanderSim.state_above_ground, [58] quat_from_axis_angle, [59] quat_from_tilt, [60] quat_mul, [61] quat_to_mat, [62] plane_height, [71] ResetOptions.from_any, [72] LanderSim.reset, [79] LanderSim._apply_mass, [83] LanderSim.hover_trim, [92] LanderSim.engines_off, [97] LanderSim.telemetry, [100] PlaneTerrain.height, [101] PlaneTerrain.height_above, [102] LanderSim.t, [103] tilt_of, [105] LanderSim._result, [107] LanderSim.read_sensors, [110] PlaneTerrain.nearest, [113] PlaneTerrain.raycast, [130] Telemetry.vz, [131] Telemetry.vh, [132] Telemetry.spin_dps, [149] LanderSim.step, [151] LanderSim._physics_step, [158] LanderSim._contacts, [160] LanderSim._flag_events
`lander/touchdown.py`	[52] TouchdownMonitor.__init__, [91] TouchdownMonitor.reset, [93] TouchdownMonitor._judge, [94] TouchdownMonitor.note_engines, [136] TouchdownReport.success, [137] TouchdownReport.failures, [163] TouchdownReport.done, [164] stability_distance, [167] TouchdownMonitor.update
`lander/vehicle.py`	[33] MassModel.__init__, [35] MassModel.total, [47] EngineBank.__init__, [48] Vehicle.__post_init__, [84] Vehicle.trim_thrust, [85] EngineBank.thrust_to_valve, [86] EngineBank.valve_to_thrust, [87] EngineBank.reset, [88] Vehicle.reset, [147] EngineBank.command, [148] Vehicle.command, [153] EngineBank.step, [154] Vehicle.step
`lander/video.py`	[66] FrameRenderer.__init__, [69] VideoWriter.__init__, [117] FrameRenderer.render, [138] _font, [139] overlay_hud, [140] VideoWriter.add, [172] FrameRenderer.close, [173] VideoWriter.close
`lander/viewer.py`	[187] watch, [189] on_key (in watch)
`lander/visuals.py`	[67] tracking_camera, [68] named_camera, [119] add_geom, [120] add_plumes, [121] decorate, [123] _frame_with_z, [124] add_arrow, [129] wind_heading_deg
`scripts/benchmark.py`	[201] _worker, [206] run, [207] benchmark.main
`scripts/check_config.py`	[4] check_config.main, [19] format_value, [20] print_table
`scripts/common.py`	[3] add_config_args, [6] config_from_args, [25] add_wind_arg, [64] wind_options, [175] summarize
`scripts/export_video.py`	[26] export_video.main
`scripts/fetch_sources.py`	[1] fetch_sources.main
`scripts/watch.py`	[179] watch.main, [186] reset_fn (in watch.main), [192] policy (in watch.main)